Duke

AI Accelerator: Analytics and Data Analysis

Jessica Nash

Jun 24, 2026

Contents

Tutorial-style lessons for AI-assisted analytics from exploration to dashboard.

How To Read This Site

Start with the first section, then progress through the remaining modules in order.

The learning material starts notebook-first with Codex in a JupyterLab terminal, then transitions to broader CLI/project workflows for reproducible dashboard delivery.

1 Pre-Workshop: Set Up Your Environment and Coding Assistant

Setup page: Open Setup

Page	Questions	Objectives
Set Up	• What do I need to complete before Session 1 so my machine is workshop-ready?	• Verify core tooling: Python 3.11+, venv, pip, Node.js, npm, and VS Code • Check that the required tools are installed and working (Python, Node.js, npm, and VS Code) • Install Codex and the connector needed for the workshop • Sign in to Codex using the method for your Duke affiliation • Set up your project environment, install the required packages, and run the final setup checklist

2 Session 1: Foundations for AI-Assisted Analytics Work

Section page: Open Session 1

Use Codex as a coding agent for analytics work while keeping the analysis understandable, reviewable, and traceable.

Page	Questions	Objectives
Coding Agents for Analytics Work	• What can a coding agent do, and what must its output satisfy?	• Identify the understandable, reviewable, and traceable requirements for agent-produced analysis.
Prompting and Project Rules	• How do prompts and an AGENTS.md file determine what Codex produces?	• Write prompts and project rules that produce reproducible notebook artifacts.
Understanding the Data Before Analysis	• What does one row represent, and which file answers which question?	• Investigate data structure before plotting or interpreting trends.
Planning and Running an Analysis	• How have physical and digital checkouts changed over time, and how can Codex help plan a new visualization?	• Select the correct dataset, account for partial years, compare visualization approaches, and trace results to code.
Managing Context in Coding Agent Conversations	• What belongs in the conversation, and what belongs in project files?	• Distinguish working context from saved project context and decide where verified facts, decisions, and workflow rules should live.
Homework: Extend the Keyword Analysis	• How can a messy keyword field be investigated with more than one model or method?	• Extend the subject-tag analysis, compare approaches, and identify what should remain human-reviewed.
Session 1 Q&A	• What questions came up during Session 1?	• Review answers to participant questions after the session.

3 Session 2: Context Management and Agent Skills

Section page: Open Session 2

Choose working file formats that reduce unnecessary agent tool use, then package repeated instructions as reusable agent skills.

Page	Questions	Objectives
Context Management Part 2	• How does the working file format change what Codex has to do?	• Explain why notebooks can create extra tool use and choose a better working artifact for agent-assisted analysis.
Agent Skills	• How can reusable instructions make agent workflows more consistent?	• Distinguish prompts, project rules, and skills, then create or evaluate a small analytics skill.

4 Session 3: From Analysis to Dashboard

Section page: Open Session 3

Use multi-agent workflows to prepare reproducible analysis outputs, preserve provenance, and turn CSV files into a stable contract for dashboard delivery.

Page	Questions	Objectives
Building Datasets with Codex	• How do you turn a public data source into a usable teaching dataset? • When should you use complete aggregate files or sampled detail files? • How do sampling choices shape the claims learners can make?	• Investigate a public API access pattern before retrieval. • Separate volume questions from composition questions. • Define row meaning, sampling strategy, reproducibility, and validation checks.
Multi-Agent Workflows	• What is a multi-agent or subagent workflow? • When should you use subagents? • How do you use subagents without losing control of the project?	• Distinguish parent, worker, and checker responsibilities. • Decide when a task is ready to delegate. • Write bounded subagent task contracts and review the results.
Build and Deploy a Static Dashboard: Prompt Notes	• How can Codex turn dashboard-ready CSV files into a static dashboard? • How can prompts change the chart, library, or dashboard component? • What steps publish a static dashboard with GitLab Pages?	• Use prompts to generate and revise a static HTML/CSS/JavaScript dashboard. • Keep dashboard data loaded from CSV files at runtime. • Follow the basic GitLab Pages deployment sequence for static assets.
Open Dataset Starting Points	• What public datasets could support a student dashboard? • Which sources have fields that support filtering, aggregation, maps, or time series? • How can you choose a dataset that leads to a clear dashboard question?	• Compare several public data sources for dashboard potential. • Select a dataset with a usable unit of analysis and meaningful dimensions. • Turn a dataset idea into a focused dashboard question.

4.1 Setup

This page will walk you through installing the required software, setting up an environment, and installing and configuring the AI agent (Codex) we will use for this workshop series.

Special Note for Windows Users

For this workshop, you will need a working Python programming environment where you can create virtual environments and run Jupyter notebooks.

If you do not already have this set up, we recommend that **Windows users use the Windows Subsystem for Linux (WSL)** for the workshop. WSL lets you run a Linux environment on Windows, which is often easier for Python development.

Most Windows users should be able to install **Ubuntu for WSL** directly from the Microsoft Store using this link: Ubuntu/WSL. After installing it, you can start your Linux shell by searching for **"Ubuntu"** in the Windows search bar.

If the Microsoft Store installation does not work, follow Microsoft's more detailed WSL installation instructions here: Install WSL.

You may also want to install Windows Terminal, which provides a nicer interface for using WSL and other command-line tools.

Download the Workshop Setup Installer

Download the workshop setup installer now, but do not run it yet. This page first walks you through computer-level setup: WSL if needed, Python, npm, Codex, and the Codex Jupyter connector.

The installer is a zip file. If you plan to unzip it from the terminal and the `unzip` command is not available, install it first. On Ubuntu/WSL, run `sudo apt update` and then `sudo apt install unzip`. On macOS, `unzip` is usually already installed; if it is missing, install the Xcode Command Line Tools with `xcode-select --install`.

Later, in the AI provider access section, you will use the installer to create the workshop-specific Python/Jupyter environment and Codex API key configuration.

Set Up Your Development Environment

In this section, you will check that the basic tools needed for the workshop are installed and available on your computer. You will run these checks from a terminal, which is the program we will use to type setup commands.

Start by opening a terminal for your operating system.

Mac

Open the **Terminal** app.

WSL/Linux

Open your Linux terminal.

- **Windows users using WSL:** open **Ubuntu** from the Windows search bar, or open **Windows Terminal** and choose your Ubuntu/WSL profile.
- **Linux users:** open your usual terminal application.

Windows users should run the commands on this page **inside the Ubuntu/WSL terminal**, not in PowerShell or Command Prompt.

Once your terminal is open, use the commands below to check that each required tool is installed.

Python

You will need **Python 3.11 or newer**.

Check your Python version with:

Mac

```
python3 --version
```

WSL/Linux

```
python3 --version
```

You should see a version number beginning with `3.11` or higher.

If Python is not installed, or if your version is older than Python 3.11, install a newer version before continuing.

Mac

Install Python from python.org or with a package manager such as Homebrew.

After installing Python, run the version check again.

WSL/Linux

Install Python inside your Linux environment. On Ubuntu/WSL, you can usually install Python with:

```
sudo apt update
sudo apt install python3 python3-venv python3-pip
```

After installing Python, run the version check again.

Python virtual environment and pip support

In addition to Python itself, you will need support for virtual environments and `pip`, Python's package installer.

Check that you can create virtual environments with:

```
python3 -m venv --help
```

Check that `pip` is available with:

```
python3 -m pip --version
```

If both commands work, you can continue.

Mac

If `venv` or `pip` is missing, reinstall Python from python.org or use your package manager to install a complete Python distribution.

WSL/Linux

If `venv` or `pip` is missing, install them inside your Linux environment:

```
sudo apt update
sudo apt install python3-venv python3-pip
```

After installing them, run the checks again.

Node.js and npm

You will also need **Node.js** and **npm** to install Codex. `npm` is the package manager used to install many command-line tools written for the JavaScript ecosystem.

Check that Node.js is installed with:

```
node --version
```

Check that `npm` is installed with:

```
npm --version
```

If both commands print version numbers, you can continue.

Mac

If Node.js or `npm` is missing, install Node.js from nodejs.org or with a package manager such as Homebrew.

After installing Node.js, run the checks again.

WSL/Linux

If Node.js or `npm` is missing, install them inside your Linux environment. On Ubuntu/WSL, you can usually install them with:

```
sudo apt update
sudo apt install nodejs npm
```

After installing Node.js and `npm`, run the checks again.

Code editor

We recommend using **Visual Studio Code (VS Code)** as your code editor for this workshop.

VS Code gives you a convenient place to edit files, open a terminal, work with notebooks, and view the workshop files.

Mac

Install VS Code from code.visualstudio.com.

You should be able to open VS Code as an application. During the workshop, it will be useful to be able to open VS Code from the terminal. Follow these instructions to enable opening code from your terminal

After it is enabled, you should be able to open VS Code using the command

```
code --version
```

WSL/Linux

Windows users using WSL: install VS Code on Windows from code.visualstudio.com, not inside WSL.

Then install the Remote Development extension packin VS Code. This allows VS Code to open and edit files inside your Ubuntu/WSL environment.

After installing VS Code and the extension, open your Ubuntu/WSL terminal and check that VS Code can be opened from the terminal:

```
code --version
```

Later, after you download and open the workshop materials, you will be able to open the project in VS Code from inside the project folder with:

```
code .
```

Linux users: install VS Code from code.visualstudio.com or use your preferred code editor.

Install Codex

Install the Codex command-line tool with npm:

```
npm install -g @openai/codex
```

If this command fails with a permissions error, use the steps in *Troubleshoot npm global installs*, then run the install command again.

You will also need to install a connector for Codex and the Jupyter Lab interface:

```
npm install -g @zed-industries/codex-acp
```

If this command fails with a permissions error, use the steps in *Troubleshoot npm global installs*, then run the install command again.

Troubleshoot npm global installs

Codex is installed as a global npm package. On some systems, global npm installs fail because the default global install folder requires administrator permissions.

> **Warning**
>
> Do not run the commands in this section unless an `npm install -g` command fails with a permissions error. If the global install command works, skip this section.

If `npm install -g` fails because npm cannot write to the global install folder, configure npm to install global packages into a folder in your home directory, then try the failed install command again.

Mac

```
mkdir -p ~/.local
npm config set prefix ~/.local
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zprofile
source ~/.zprofile
```

WSL/Linux

```
mkdir -p ~/.local
npm config set prefix ~/.local
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.profile
source ~/.profile
```

Then check that the `codex` command is available:

```
codex --version
```

If your terminal says that `codex` was not found, close and reopen your terminal, then try the version check again.

AI provider access

This workshop uses **Codex**, OpenAI's coding agent. You will be using Codex both in the JupyterLab interface with the JupyterAI plugin and in the command line interface. Some Duke departments cover access to ChatGPT. If you are able to get ChatGPT access through Duke without paying out of pocket, please set it up through Duke Software Manager before the workshop.

If your department does not cover access, you do not need to pay for ChatGPT for this workshop unless you want to. We will also provide a workshop API key that you can use instead; *skip to the workshop API key setup*.

Open Codex by typing `codex` into your terminal.

The first time you open Codex, choose the first option, **Sign in with ChatGPT**. This will open a web page where you should log in to ChatGPT using your Duke NetID.

> **Note**
>
> If you cannot get ChatGPT access, or if access would require you to pay and you do not want to, use the workshop API key setup below.

After logging in, check your login status:

```
codex login status
```

If ChatGPT login works, you can use Codex normally.

```
codex exec "Reply with only: Codex is working" --skip-git-repo-check
```

Workshop API key setup

We will provide API keys to all workshop participants. Set up the workshop API key configuration even if ChatGPT login works for you. This gives you a backup provider if ChatGPT login is unavailable or if you run out of usage during the workshop.

After you have installed Python, npm, Codex, and the Codex Jupyter connector, use the workshop setup installer. The zip includes a `README.md`, a `requirements.txt` for testing JupyterLab and Jupyter AI, and a script that creates the workshop Codex LiteLLM configuration after you paste in your API key.

Move the downloaded zip file to the folder where you want to create your workshop setup, or retrieve it again from the terminal with `curl`.

```
curl -L
↪'https://github.com/AI-for-Data-Analysis/ai-for-data-analysis.github.io/raw/refs/heads/main/files/s
↪' -o student-setup.zip
unzip student-setup.zip
```

> **Tip**
>
> In WSL, you can open the current Linux folder in Windows File Explorer with:
>
> ```
> explorer.exe .
> ```
>
> You can also open a specific path by replacing `.` with that path.

Then run the Codex API key setup script:

```
cd student-setup
python3 scripts/setup_codex_litellm.py
```

When prompted, paste the API key provided by the workshop instructors.

The script creates a separate Codex home directory named `~/.codex-litellm`. Codex will still use your regular `~/.codex` folder by default and will use `~/.codex-litellm` only when you explicitly launch it that way.

Then test the workshop API key configuration:

```
CODEX_HOME="$HOME/.codex-litellm" codex exec "Reply with only: Codex is working" --
↪skip-git-repo-check
```

Optional shortcut

A shell function is a shortcut in your terminal. The function below creates a command named `litellm` that runs one command with the workshop Codex configuration.

This does not permanently set `CODEX_HOME`. It sets `CODEX_HOME` only for the command you run after `litellm`.

Mac

```
cat >> ~/.zshrc <<'EOF'
litellm() {
  CODEX_HOME="$HOME/.codex-litellm" "$@"
}
EOF
source ~/.zshrc
```

WSL/Linux

```
cat >> ~/.bashrc <<'EOF'
litellm() {
  CODEX_HOME="$HOME/.codex-litellm" "$@"
}
EOF
source ~/.bashrc
```

After adding the function, you can test Codex with:

```
litellm codex exec "Reply with only: Codex is working" --skip-git-repo-check
```

During the workshop, use normal Codex for ChatGPT login when available. Launch commands with `CODEX_HOME="$HOME/.codex-litellm"` when you need the workshop API key.

Set Up the Student Installer Environment

Use the workshop setup installer to verify that your environment can run JupyterLab, Jupyter AI, and the workshop Codex API key configuration.

Create a virtual environment

The setup installer contains a `requirements.txt` file with the Python packages needed for the setup check. We will use a Python virtual environment for these packages.

You should execute the following commands from the `student-setup` folder. Create a virtual environment for the setup check:

Recommended shortcut

The installer includes a script that creates `.venv`, activates it inside the script, upgrades `pip`, and installs `requirements.txt`.

```
bash scripts/setup_python_environment.sh
```

After the script finishes, activate the environment in your current terminal before running more commands:

```
source .venv/bin/activate
```

If you prefer to run the steps manually, create the virtual environment with:

```
python3 -m venv .venv
```

Then activate the virtual environment:

Mac

```
source .venv/bin/activate
```

WSL/Linux

```
source .venv/bin/activate
```

After activation, you should see `(.venv)` at the beginning of your terminal prompt.

Install Python packages

Install the required Python packages from `requirements.txt`:

```
python -m pip install -r requirements.txt
```

Checklist

Before the workshop, open your terminal and run these checks to confirm your setup.

Basic system checks

```
python3 --version         # should show Python 3.11 or newer
python3 -m venv --help    # should show venv help text
python3 -m pip --version  # should show a pip version
node --version            # should show a Node.js version
npm --version             # should show an npm version
```

Codex command-line checks

First check that the Codex command-line tool is installed:

```
codex --version           # should show a Codex version
codex-acp --help          # should show codex-acp help text
```

If you were able to sign in with ChatGPT, you can also test normal Codex access:

```
codex exec "Reply with only: Codex is working" --skip-git-repo-check # should return:␣
↪Codex is working
```

If you cannot sign in with ChatGPT, skip that normal Codex access test and use the workshop API key test instead:

```
CODEX_HOME="$HOME/.codex-litellm" codex exec "Reply with only: Codex is working" --
↪skip-git-repo-check # should return: Codex is working
```

If you added the optional `litellm` shell function, you can run the same check with:

```
litellm codex exec "Reply with only: Codex is working" --skip-git-repo-check # should␣
↪return: Codex is working
```

Run from the workshop materials folder in your terminal

Move into the folder where you downloaded and unzipped the workshop materials. Replace `path/to/workshop-materials` with the actual path to your folder.

```
cd path/to/workshop-materials
```

Check that the expected files and folders are present.

```
ls                  # should show the workshop files
ls .venv            # should show the virtual environment folder contents
ls requirements.txt # should show requirements.txt
```

Activate the virtual environment.

```
source .venv/bin/activate
```

After activation, run these checks.

```
python --version        # should show Python 3.11 or newer
python -m pip --version # should show pip from the virtual environment
python -m pip check     # should show: No broken requirements found.
jupyter lab --version   # should show a JupyterLab version
```

Workshop Data

Download the workshop data zip and put it inside a folder named `analytics-accelerator`.

Use this link if you want to download the zip in your browser: student materials

After you've downloaded the material, move to the `student-setup` folder inside of your `analytics-accelerator` folder. Then, unzip the file.

You can alternatively use terminal commands, run:

```
cd ~/analytics-accelerator
cd student-setup
curl -L
↪'https://github.com/AI-for-Data-Analysis/ai-for-data-analysis.github.io/raw/refs/heads/main/files/s
↪' -o seattle-public-library.zip
unzip seattle-public-library.zip
```

After unzipping, you should have:

```
~/analytics-accelerator/student-setup/seattle-public-library/
```

4.2 Session 1: Foundations for AI-Assisted Analytics Work

This session uses Codex, a coding agent, to investigate a dataset and answer a question about it while keeping the analysis **understandable, reviewable, and traceable**. The example dataset is two decades of library checkout records.

The session covers how prompts and an `AGENTS.md` rules file determine what Codex produces, how to establish what the data represents before analyzing it, and how to produce a trend analysis in which each result is traceable to its code.

Complete the environment and assistant setup before starting: Complete setup

Page	Questions	Objectives
Coding Agents for Analytics Work	• What can a coding agent do, and what must its output satisfy?	• Identify the understandable, reviewable, and traceable requirements for agent-produced analysis.
Prompting and Project Rules	• How do prompts and an AGENTS.md file determine what Codex produces?	• Write prompts and project rules that produce reproducible notebook artifacts.
Understanding the Data Before Analysis	• What does one row represent, and which file answers which question?	• Investigate data structure before plotting or interpreting trends.
Planning and Running an Analysis	• How have physical and digital checkouts changed over time, and how can Codex help plan a new visualization?	• Select the correct dataset, account for partial years, compare visualization approaches, and trace results to code.
Managing Context in Coding Agent Conversations	• What belongs in the conversation, and what belongs in project files?	• Distinguish working context from saved project context and decide where verified facts, decisions, and workflow rules should live.
Homework: Extend the Keyword Analysis	• How can a messy keyword field be investigated with more than one model or method?	• Extend the subject-tag analysis, compare approaches, and identify what should remain human-reviewed.
Session 1 Q&A	• What questions came up during Session 1?	• Review answers to participant questions after the session.

Coding Agents for Analytics Work

This session uses Codex, a coding agent from OpenAI, to investigate a dataset and answer questions about it while keeping the work reproducible. This page covers the workspace and the requirements your analysis must meet before you write the first prompt.

Download the student materials before starting

Create a folder for this lesson, put the student materials zip inside it, unzip the materials, and move into the unzipped project folder before starting JupyterLab or Codex.

If you use the terminal, run:

```
cd ~/analytics-accelerator/
cd student-setup
curl -L
↪'https://github.com/AI-for-Data-Analysis/ai-for-data-analysis.github.io/raw/refs/heads/main/files
↪' -o seattle-public-library.zip
unzip seattle-public-library.zip
```

If you downloaded the zip in your browser instead, move `seattle-public-library.zip` into your

`analytics-accelerator` folder before unzipping it.

Workspace

The setup step prepares everything required. For this session:

- **JupyterLab** is the working environment.
- The **notebook** is the analysis artifact: the file you keep, rerun, and share.
- **Codex** is the coding agent, run from a terminal inside JupyterLab.
- The project contains a **`data/`** folder with the files to analyze.

Codex's work should run in the project environment created during setup and be written into the notebook.

For the workshop, we will use Codex in a JupyterLab terminal. The notebook remains the analysis artifact, but the agent interaction happens in the terminal. This gives participants one consistent place to approve actions, see commands, and decide when to start a fresh session.

Opening JupyterLab and Codex

To open JupyterLab, you should navigate to the directory where you would like to work using your terminal. You can then launch JupyterLab using the `jupyter lab` command:

```
cd ~/analytics-accelerator/
cd student-setup
source .venv/bin/activate
ls
```

You should see your `seattle-public-library` folder.

Next, start JupyterLab

```
jupyter lab
```

When JupyterLab opens, it usually starts on the launcher screen. The launcher is the starting point for creating new notebooks, opening terminals, and accessing project files.

In the screenshot, **A** marks the Python notebook tile. Use this tile to create a new notebook for the analysis. The notebook is where the code, outputs, notes, and final analytical steps should live.

For this workshop, open a **Terminal** from the launcher or the JupyterLab menu, then start Codex in the project folder. The exact command may depend on your setup, but the workflow is the same: Codex runs in the terminal, and the notebook records the analysis.

Arrange the terminal and notebook side by side. You can do this by clicking and dragging the windows. This keeps the agent interaction visible while you review the code and output in the notebook.

You can open Codex by typing

```
codex
```

into your terminal.

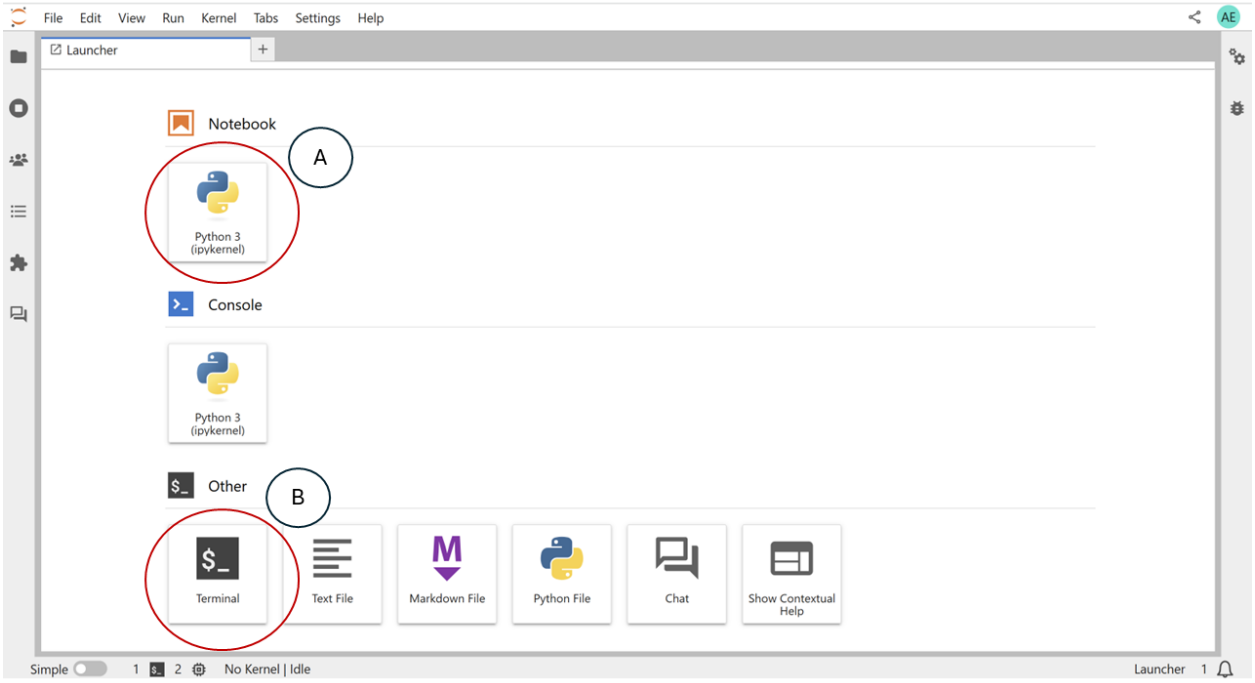

Fig. 1: The JupyterLab launcher is where you open the notebook and terminal workspace.

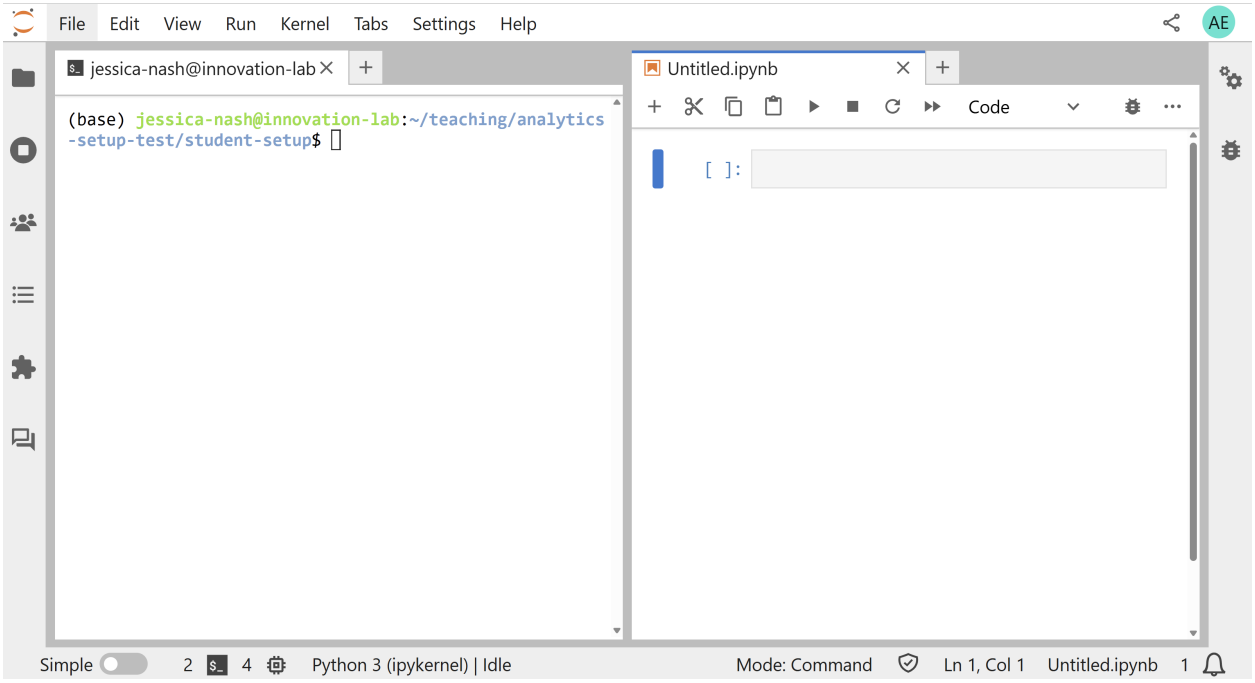

Fig. 2: Keep the terminal and the analysis notebook visible at the same time in the JupyterLab workspace.

Codex terminal basics

In the Codex terminal, most of what you type is a normal prompt. Messages such as "profile the data in `seattle-public-library/`" or "add a short notebook section explaining this chart" are sent to the agent as work requests.

Codex also has **slash commands** for controlling the session. Slash commands start with `/` and are handled by the Codex interface rather than treated as analysis prompts.

Useful commands to know:

- `/status` shows session and environment information.
- `/model` lets you inspect or change the model for the session.
- `/new` starts a fresh session.
- `/compact` summarizes the current conversation to reduce context.
- `/quit` exits Codex.

Prompt or command?

If you want Codex to do analysis work, write a normal prompt. If you want to control the Codex session itself, use a slash command.

Exercise: Prepare the workspace

Open JupyterLab and arrange a terminal and a new Python notebook side by side.

Before continuing, make sure you can identify:

- where you will type prompts to Codex
- where Codex should record code and outputs
- where the project files and `seattle-public-library/` folder are located

What a coding agent does

A coding agent is an AI assistant connected to a working project environment. Codex can read files, write Python, run that Python, edit files, and produce plots in the course of answering one request. It can change the project rather than only describing it.

This raises a requirement that does not apply to a chat assistant. When the output is code and analysis, that output must be possible to follow and verify because the answer depends on specific data, transformations, calculations, and assumptions. As we will see in the lessons, by default, Codex may run calculations in a temporary terminal, report a conclusion, and leave no corresponding code in the notebook. The result cannot then be checked or reproduced.

Requirements for the analysis

Each analysis in this session must be:

- **Understandable** — the code and explanations can be read and followed.
- **Reviewable** — analytical steps are separated clearly enough to check individually.
- **Traceable** — every stated result corresponds to specific code and output.

The process

The work proceeds as an iterative cycle rather than a single prompt:

- Run a prompt.
- Review what Codex did.
- Identify what was missing or incorrect.
- Record a rule to correct it.
- Run again, then continue into the analysis.

The remaining pages apply this cycle to the checkout dataset: an initial broad prompt, a set of project rules, a structured investigation of the data, and a trend analysis.

Key points

- A coding agent works inside a project environment, not only in a chat window.
- The notebook is the analysis artifact students should be able to rerun and review.
- Agent-produced analysis must be understandable, reviewable, and traceable.

Prompting and Project Rules

A **prompt** is an instruction or message given to an AI system. It can ask a question, request an action, provide context, point the AI toward specific files, or tell it what kind of output to produce. In this lesson, prompts are the messages you type to Codex in the JupyterLab terminal.

Working with a coding agent is often iterative, especially at the beginning of a project. You ask for something, observe what the agent does, and then refine either the next prompt or the standing project instructions. The first prompt does not need to be perfect, but each response gives you evidence about what the agent needs to be told more explicitly.

As the project develops, the interaction may become less repetitive. Once expectations are recorded in `AGENTS.md` and the notebook contains more context, later prompts can often be shorter because Codex has more project guidance to work from.

The wording of a prompt determines what Codex does for the immediate task. A project rules file determines how Codex should work across the project. In this lesson, we first observe what Codex does with broad and notebook-focused prompts, then turn those observations into starter rules in `AGENTS.md`.

Sessions are separate contexts

Each new Codex session starts a new conversation context. Codex does not automatically know what you discussed in a different session unless that information is available in the project files, the open notebook, or the current conversation.

This is useful when you are trying different things. For example, you might use one session to investigate the data, another session to revise project rules, and another session to work on a specific visualization. Separate sessions help keep those tasks from blurring together.

The tradeoff is that context does not magically carry over. If a decision matters across sessions, record it somewhere durable: in the notebook, in `AGENTS.md`, in a README, or in another project file. Treat the session as the working conversation and the project files as the shared memory.

In the first part of this lesson, we intentionally try several prompts in different sessions. That makes the context boundary visible: each new session may need to inspect the repository and data again before it can answer well. That repeated orientation is not a mistake, but it is work we can reduce once the project starts to take shape. As the project develops, we can write important context into project files and record standing expectations in `AGENTS.md`.

The goal is not to prevent Codex from working quickly. The goal is to make sure its speed produces analysis that can be inspected, revised, and rerun. The workflow is:

- Try a prompt.
- Observe what Codex does.
- Identify what should be more reviewable.
- Record standing expectations in `AGENTS.md`.
- Try again with those expectations in place.

Try one broad prompt before rules

Start with a broad request before adding project rules. This makes Codex's default behavior visible.

> **Codex in the JupyterLab terminal**
>
> In the terminal workflow, type the prompt directly into Codex. Do not include `@Codex`; that mention was only needed in the earlier Jupyter chat workflow.

In the Codex terminal, enter:

```
I need to do exploratory data analysis on the data in my seattle-public-library␣
↪folder. Can you investigate and tell me about it?
```

Watch what Codex does before evaluating the answer. Note whether it writes reusable notebook code, works in the terminal, summarizes in the response, creates files, or makes assumptions about the data.

What Codex did

> **Note**
>
> Because coding agents use LLMs and can choose different valid paths through a task, your run may not match this sequence exactly. You may see different commands, a different order of operations, or slightly different wording in the final response. The important pattern is whether Codex leaves behind work that is easy to inspect and rerun.

In testing, Codex inspected the project structure and the files in `data/`. Your run might do this in a different order, but it should find three CSVs:

```
combined_checkout_totals_by_month_usageclass.csv
digital_checkout_title_sample_balanced_250k.csv
physical_checkout_title_sample_balanced_250k.csv
```

Codex might run file and environment checks, including commands such as:

```
pwd
List data
Read digital_checkout_title_sample_balanced_250k.csv
Read physical_checkout_title_sample_balanced_250k.csv
Read combined_checkout_totals_by_month_usageclass.csv
wc -l data/*.csv
```

It then might write and execute Python code in the terminal to compute:

```
dataset shapes
columns and dtypes
missingness
date ranges
usageclass counts
checkouttype counts
materialtype counts
checkout summaries
top creators
top publishers
top titles
top subject tokens
rows by year
yearly totals and shares
```

What Codex reported

Codex might produce a polished EDA summary. In testing, it identified:

```
combined_checkout_totals_by_month_usageclass.csv: 501 rows, monthly aggregate data␣
↪from April 2005 through April 2026.
digital_checkout_title_sample_balanced_250k.csv: 250,000 sampled digital title-month␣
↪rows.
physical_checkout_title_sample_balanced_250k.csv: 250,000 sampled physical␣
↪title-month rows.
```

It might also surface findings such as:

```
Physical checkouts dominated from 2005 through 2019.
Digital grew gradually and became the majority in 2020.
The two title-level files are balanced samples, not full raw extracts.
The aggregate file should be used for full checkout volume trends.
```

What this demonstrates

Codex may do a lot from a vague prompt. That is useful for quick orientation, but the work may not be very traceable. Much of the analysis may happen in temporary terminal code, and the final output may be a response summary rather than a reusable notebook artifact.

The broad prompt is useful here as a teaching demonstration, but it is not the most efficient way to work. It may spend tokens on a wide scan, a long summary, or analysis steps that do not become part of the notebook. The request also did not specify where the work should be recorded, ask Codex to keep the code simple, explain the data structure before interpreting trends, or leave behind a notebook section that another person could review.

Try a notebook prompt

Next, ask Codex to put the investigation into the notebook:

```
Can you help me investigate the data in my seattle-public-library folder? I would␣
↪like to know what the dataset or datasets are and the relationships between the␣
↪items.
```

(continues on next page)

(continued from previous page)

```
I have a notebook open. Please add a brief Data Inventory section that previews each␣
↪file, summarizes the rows and columns, and explains what one row seems to represent.
```

This prompt demonstrates a capability that response summaries do not show: Codex can edit the notebook itself, adding code cells and explanatory text instead of only answering in the terminal.

For this activity, the desired notebook shape is intentionally basic: a short markdown setup, a code cell that previews the data files, a compact summary table, and a few cautions about interpretation. The prompt does not spell all of that out, so the class can observe whether Codex makes reasonable choices or adds too much.

It also gives us something important to inspect: the kind of notebook code Codex might write before we give it project rules. Even with this basic request, the code may run correctly but still be difficult to review. For example, Codex may combine several analytical steps into one compact pandas expression, use helper columns without explaining them, or write more explanation than the notebook needs.

You may also see Codex do substantial profiling in the terminal before it edits the notebook. That can be useful scratch work, but it is a problem if the important reasoning only appears in terminal output or a response summary. For this course, the durable artifacts should be the notebook and project files. If Codex keeps rediscovering the same dataset facts in scratch commands, that is a signal to write those facts down where future sessions can find them.

However, this prompt still leaves important interpretation questions implicit. It asks about "datasets" and "relationships," but it does not explicitly require Codex to answer:

- What does one row represent?
- What is being counted?
- Is each file sampled, aggregated, or complete?
- What would be misleading to infer from each file?

The narrower prompts improve the immediate task, but they do not solve the whole workflow problem. We still need project rules that tell Codex what reviewable notebook code should look like and require it to clarify the data structure before analysis.

These first prompts motivate the need for rules. We do not need to wait until the notebook is messy to define expectations. After observing Codex with broad and narrower prompts, we can already see that it needs guidance about simple code, traceable work, and understanding the data structure before analysis.

Exercise: Draft rules from observation

In your group, review what Codex did across the first prompts. Draft three rules that would make its analytics work easier to inspect, revise, or rerun.

For each rule, write down the observation that motivated it. For example:

- Observation: Codex answered in the terminal but did not leave notebook code.
- Rule: Put investigation code and short explanations in the open notebook.

Keep the rules short. You will compare them with the starter `AGENTS.md` rules in the next section.

Show how AGENTS.md works

`AGENTS.md` is a project instruction file: a plain Markdown file that lives in the project folder and gives coding agents standing instructions for how to work in that project. It is not a Python file, a notebook file, or a data file. It is also not unique to this dataset; it is an AGENTS.md convention used by coding-agent tools to find project-specific guidance.

In this workshop, we use `AGENTS.md` to tell Codex how to handle analytics work in this repository. Once Codex has been asked to write in a notebook during a session, it may continue doing that within the same conversation. But if notebook-based, traceable analysis is an expectation for the whole project, we do not want to retype that instruction at the start of every new session. `AGENTS.md` gives us a place to record those standing expectations so they can be loaded automatically.

Create a file called `AGENTS.md` and use a temporary rule to make the instruction mechanism visible. Add this section to `AGENTS.md`:

```
## Response Style Test

Answer in haiku.
```

In codex, open a new thread using the `/new` command.

Ask Codex a simple question and observe whether the response changes. If the response is formatted as a haiku, the project rules are reaching the agent.

Then remove the haiku rule. The haiku test is not part of the analytics workflow; it simply shows that Codex is reading project instructions.

Start a new session after changing AGENTS.md

After changing `AGENTS.md`, start a new Codex session before testing the new rule. Existing sessions may already have loaded the previous instructions. Starting a new session is the clearest way to make sure Codex reads the updated file. This is another example of the same context principle: a new session starts fresh, then loads the current project instructions.

Add first-round AGENTS.md rules

Now replace the temporary rule with a small set of non-negotiable rules for analytics work. Later, groups will add their own rules based on what they observe.

After updating `AGENTS.md`, start a new Codex session again so the starter analytics rules are loaded for the next prompt.

Update `AGENTS.md` with the starter rules below, and fill in the Python experience level for your audience.

```
# AGENTS.md

## Purpose

This project is for an AI-assisted analytics workflow.

We are using Codex to help investigate data, write Python code, create notebook␣
↪explanations, and build reproducible analysis artifacts.

## Audience and Experience Level

Notebook code should be written for a student with one semester of Python
experience. Use explicit, step-by-step pandas code with descriptive intermediate
variables. Avoid compact idioms, dense method chaining, and unexplained
reshaping or aggregation shortcuts.

## Non-Negotiable Rules

```

(continues on next page)

(continued from previous page)

```
1. Treat the notebook as the durable analysis record, not as a full transcript of␣
↪scratch work. For accepted analysis, write the code, outputs, checks, and short␣
↪explanations needed to understand and rerun the result.

2. Inspect actual files and data before making claims, but do not repeatedly␣
↪re-investigate the same data when the notebook already contains sufficient evidence.

3. If a new question depends on information not yet established in the notebook, add␣
↪only the minimal new inspection needed to answer it.

4. Do not run separate terminal or scratch Python analysis to preview, verify, or␣
↪compute results that belong in the accepted notebook analysis. Terminal commands␣
↪may be used for non-analytical maintenance, such as file existence checks, notebook␣
↪structure checks, and environment checks.

5. Do not modify raw data files.

6. Do not duplicate accepted analysis work. If a table, chart, statistic, join, data␣
↪profile, or interpretation will support a notebook claim, compute it in the␣
↪notebook only.

7. Make transformations explicit, readable, and rerunnable.

8. Do not invent column names, file names, metadata, or dataset meanings.

9. Keep the notebook clean. Include durable inspection, analysis, transformations,␣
↪checks, outputs, and short explanations that support the work. Do not include␣
↪irrelevant experiments, repeated previews, or exploratory dead ends unless they␣
↪explain an important decision.

10. Before creating plots or interpreting trends, identify what one row represents,␣
↪what value is being counted or aggregated, and whether the data is sampled,␣
↪aggregated, or complete.
```

`AGENTS.md` applies to the whole project and is loaded automatically when a new agent session starts. A prompt shapes the immediate task; `AGENTS.md` records the standing instructions you want Codex to carry into each new task in this project.

Exercise: Add and test your own rule

Add one small rule to `AGENTS.md` that would make Codex's notebook work easier for you to review. Start a new Codex session, then ask Codex a short question or revision request that should reveal whether the rule is being followed.

After testing, decide whether to keep, revise, or remove the rule.

With the starter rules in place, the next step is to ask Codex to create the first serious notebook artifact: an investigation of what the data represents.

Key points

- A prompt shapes the immediate task.

- `AGENTS.md` records standing project instructions that can load automatically in new agent sessions.
- Starter rules should respond to observed agent behavior, not abstract preferences.
- The broad prompt is useful because it shows why traceable notebook work and data-structure checks need to be explicit.

Understanding the Data Before Analysis

A coding agent will generate summaries and plots for any input, including data that cannot support the intended conclusion. The person doing the analysis is responsible for establishing what the data represents before interpreting results. This page covers how to direct Codex to characterize a dataset, and how to review what it returns.

Direct the investigation with specific questions

A general request ("tell me about this data") returns whatever the agent decides is relevant. A more reliable approach is to specify the questions that characterize any dataset and require the agent to answer them from the data:

- What does one row represent?
- What value is being counted or aggregated?
- Is the data a sample, an aggregate, or a complete extract?
- How do the datasets relate to one another?
- Which dataset answers which kind of question, and what would be misleading to infer from each?

These questions are the goal of the investigation, but they do not all need to go into the first prompt. Start with a smaller profiling task so Codex creates a simple notebook artifact before moving into interpretation.

Ask Codex to create the first serious notebook artifact for this project:

```
Add a simple data profile to my notebook for the data in seattle-public-library/.␣
↪Load the files, print the first few rows, report non-null values and data types.
```

After Codex finishes, review the notebook section it created. Use the profile to start asking what the files represent, but do not move on to plotting until the data structure is clear.

Two parts of this prompt are doing specific work. "Don't make any plots yet" stops the agent from producing charts before the structure is confirmed. The request asks for basic profiling outputs rather than interpretation, which helps keep the first notebook section compact and reviewable.

Why this prompt is more specific

The previous lesson used a narrower prompt:

```
Can you help me investigate the data in my data folder? I would like to know what the␣
↪dataset or datasets are and the relationships between the items.
```

That prompt was better than broad EDA because it focused on data inventory and relationships. In testing, Codex identified the three datasets and explained that the title-level files connect to the aggregate table by year, month, and usage class.

However, later testing showed that asking for all of the interpretation at once made Codex write too much code and too much explanation. We still need to answer the data-shape questions, but it is better to build up to them from a simple profile:

- What does one row represent?

- What is being counted?
- Is this sampled or aggregated?
- What would be misleading to infer?

The revised prompt asks only for the facts needed to begin that discussion: files, rows, columns, data types, and non-null counts.

Confirm the profile

Codex may add a notebook section with a compact profile for each file. Treat the profile as a starting point, not a complete understanding of the data. The next interpretive question is whether each file is a sample, an aggregate, or a complete extract, because that determines which questions the file can legitimately answer.

Review whether the notebook includes:

- files present
- row counts and shapes
- column names
- data types
- non-null counts
- a small preview of each file, if useful

After reviewing the profile, use the column names and sample rows to discuss:

- What does one row represent in each file?
- What value is being counted or aggregated?
- Which files appear sampled, aggregated, or complete?
- How might the files relate to each other?

In the checkout example, the full investigation eventually distinguishes two title-level *sample* files from one monthly *aggregate* file, and establishes that the samples relate to the aggregate by checkout year, month, and usage class. The practical consequence is immediate: a question about total volume must use the aggregate, not the samples.

In an earlier test, Codex identified:

Digital title sample

- `digital_checkout_title_sample_balanced_250k.csv`
- 250,000 title-month rows
- digital checkouts
- covers January 2006 through December 2025
- includes title metadata and checkout counts

Physical title sample

- `physical_checkout_title_sample_balanced_250k.csv`
- 250,000 title-month rows
- physical checkouts
- covers January 2006 through December 2025
- has the same schema as the digital file

Combined monthly totals

- `combined_checkout_totals_by_month_usageclass.csv`
- 501 monthly aggregate rows
- covers April 2005 through April 2026
- aggregated by `checkout_year`, `checkout_month`, and `usageclass`
- includes `total_checkouts`, `month_total_checkouts`, `usageclass_share`, `title_month_rows`, and `sample_rows`

Codex also tested the relationship between files:

```
combined_checkout_totals_by_month_usageclass
  keyed by: checkout_year + checkout_month + usageclass
        |
        | matches
        v
digital_checkout_title_sample_balanced_250k
physical_checkout_title_sample_balanced_250k
  keyed by: checkoutyear + checkoutmonth + usageclass
```

It reported that the aggregate file's `sample_rows` column matched the number of rows in the title-level sample files for each month and usage class.

Document the data for future sessions

At this point, Codex has done useful investigation that future sessions should not need to repeat from scratch. The notebook records the analysis process, but the basic dataset facts also belong in a durable reference file. Dataset documentation belongs somewhere humans and agents would both expect to find it: inside the `seattle-public-library/` folder.

Ask Codex to create a data README:

```
Please edit the README.md file in the data folder documenting the raw data files with␣
↪enough detail that future sessions do not have to rediscover the basic dataset␣
↪structure every time.

Update AGENTS.md to direct to this file.
```

Review `seattle-public-library/README.md` before keeping it. Check that Codex did not invent dataset meanings, overstate what the data can support, or turn sampled data into full-population claims.

Then ask Codex to update `AGENTS.md` so future assistants know to use the data documentation before starting new analysis:

```
Please update AGENTS.md to tell future assistants to read␣
↪seattle-public-library/README.md before analyzing the raw data files.
```

This is an important teaching moment. If Codex has to inspect the same files, schemas, row counts, and relationships in every new session, the project is missing shared memory. Once those facts have been verified, `seattle-public-library/README.md` can record them so later sessions can start from known dataset orientation and spend their work on the new question.

Request a relationship diagram

Once the relationships are established, have the agent express them visually so they can be checked at a glance:

Tip

Mermaid is a text-based diagram syntax. Instead of drawing boxes and arrows by hand, you write a small block of structured text, and a renderer turns it into a visual diagram. JupyterLab can render Mermaid diagrams in notebooks, and Codex can write Mermaid syntax for common diagram types.

In this lesson, we use Mermaid to make the dataset relationships visible. This is not a separate charting library for analyzing values; it is a way to document structure.

Ask Codex to summarize the dataset relationships visually:

```
Based on the investigation, add a Mermaid diagram to the notebook showing the
datasets and how they relate. It should look like an Entity Relationship diagram.
```

Review the diagram as an interpretation of the investigation, not as an automatically correct schema.

Note

If you want to export a Mermaid diagram outside the notebook, the official Mermaid CLI can render Mermaid text to SVG, PNG, or PDF. Mermaid CLI is a Node.js tool installed with `npm`, not a Python package. If Node.js and npm are available, one option is:

```
npm install -g @mermaid-js/mermaid-cli
```

Then this command converts a Mermaid file to SVG:

```
mmdc -i input.mmd -o output.svg
```

If you do not want to install it globally, see the Mermaid CLI documentation for local, Docker, and other installation options.

Data relationship diagram

The diagram shows a many-to-one relationship for context only: many sampled title-month rows can correspond to one monthly aggregate row. The aggregate keys are not title identifiers.

Compare the notebook investigation with the relationship diagram.

Discuss:

- Does the diagram make the sample-versus-aggregate distinction visible?
- What relationship is shown?
- What relationship is not shown?
- Which file should answer questions about total checkout volume?
- Which files should answer questions about titles, creators, subjects, or material types?

Make another diagram

Ask Codex what other types of visualizations it could make for data relationships. Pick one or describe your own and add it to the notebook.

Key points

- Understanding what one row represents comes before plotting or interpreting trends.
- Sample files and aggregate files answer different kinds of questions.
- Reviewing the agent's notebook output is part of the analysis workflow, not a separate cleanup step.

Planning and Running an Analysis

This page applies the preceding steps to a complete analysis question:

> How have physical and digital checkouts changed over time?

It also shows how to use Codex for planning before implementation when the next analysis might require a library or visualization approach we have not already chosen.

Select the appropriate file

Determine which file can answer the question before writing a prompt. The dataset contains two title-level samples and one monthly aggregate. Overall borrowing volume over time is a question about totals, so it must use the 501-row aggregate file. The 250,000-row sample files describe patterns and do not represent total volume.

This is the purpose of the previous investigation: knowing the structure of the data determines which file is correct. State the chosen dataset in the prompt.

Run the analysis

In the Codex terminal, enter:

```
Using the monthly aggregate checkout dataset, analyze how physical and digital
checkouts changed over time. Add traceable code and short explanations to the
notebook. Include annual totals, a trend chart, and a note about any partial
years.
```

When Codex finishes, use the review checklist below before accepting the result.

The prompt does not restate the requirements defined in `AGENTS.md`, such as readable code and notebook output; those apply automatically. It specifies only what is particular to this task: the dataset, the required outputs, and the caveat to address.

Expected output

Codex adds annual totals split by physical and digital, a trend chart across the full range, and the digital share over time. The data shows physical checkouts higher in the early years, digital checkouts increasing over time, and a crossover around 2020.

The aggregate file runs through April 2026, so the final year is incomplete. If 2026 is summed and charted alongside full years, it will appear as a sharp decline because four months are compared against twelve. The analysis must flag the partial year. Confirm that Codex did so; if it did not, instruct it to add the note.

Ask for a change on the chart

Choose something you would like to see changed on the chart. This could have to do with font size, color, or style.

Ask Codex to change your chart. Continue prompting until you are satisfied with the plot style.

Review checklist

Review the output against the following:

- Did it use the aggregate file rather than a sample file?
- Did it state what one row represents before interpreting the data?
- Did it account for the partial 2026 year?
- Does each stated result correspond to specific code and output?

Plan a new visualization before implementing

Not every prompt needs to ask Codex to make changes immediately. When the next step involves choosing an approach, a library, or a visualization type, use Codex as a planning partner first.

> Does the data show any seasonal patterns for checkouts over time?

This question still uses the monthly aggregate file, but the best visualization is less obvious than a basic trend line. A heatmap, faceted chart, small multiples, or an interactive chart might each work. Before installing anything or adding notebook cells, ask Codex to discuss possible approaches:

```
I'd like to discuss investigating checkout seasonality over time for checkouts. I'm␣
↪curious if
there is any time of year where checkouts occur more or less often. How might we do␣
↪that?
```

A key phrase used here is "I'd like to discuss…". This will keep Codex from acting until you have discussed the topic and greenlighted an action.

Review the options before moving forward. The useful output is not just the name of a chart type; it should explain tradeoffs such as readability, complexity, whether the chart will render reliably in the notebook, and whether the code will be easy to review.

If the recommendation is appropriate, then ask Codex to implement the chosen approach:

```
Use the recommended approach to add a simple, readable notebook section
showing monthly checkout seasonality by usage class. Explain what the
visualization shows and include one caveat.
```

This demonstrates a different prompting pattern: discuss and decide first, then implement. That pattern is useful whenever the analysis goal is clear but the method is still uncertain.

Apply the workflow to the sample data

The aggregate file answers questions about totals. The title-level sample files answer more granular questions about titles, subjects, material types, and publishers. Select one such question and apply the same workflow:

```
What material types dominate the digital vs physical samples?
Which subjects appear most often in each sample?
```

(continues on next page)

(continued from previous page)

```
What titles or creators appear in both the digital and physical samples?
How does publication year differ between digital and physical samples?
```

Use a prompt that carries the same requirements:

Group activity: Investigate a sample-data question

Choose one granular question about the title-level sample files, or write your own. Use this prompt template:

```
Help me investigate this question: [QUESTION]. Use the title-level sample
files. Before analyzing, state what one row represents and what is being
counted. Add traceable, readable code to the notebook with short explanations.
End with one finding and one caveat.
```

Prepare a short share-out with:

- the question your group investigated
- the dataset used
- what one row represents
- the code or output that supports the finding
- one finding
- one caveat or uncertainty
- one thing Codex did well or poorly

These files are samples, not the complete collection. A finding describes the sample; any statement about the full collection is an inference. State which one applies.

Stretch exercise: Treat subject tags as a text-cleaning problem

The title-level sample files include a `subjects` field. This field can look simple at first, but it is a useful stretch case because a small profiling question can turn into a text-cleaning and interpretation problem.

Start by asking Codex a narrow verification question:

```
Do the checked out materials have subject tags in any dataset? Check the raw
CSV headers and report which files include the field.
```

If the field exists, ask Codex to add a short notebook section:

```
Add a cell to the notebook that summarizes the subjects field for the digital
and physical title-level sample files. Count the number of unique subject
values in each dataset and show a small sample of values. Keep the code simple
and explain any parsing assumptions.
```

Review the result carefully. In testing, the digital and physical samples had very different unique subject counts. That difference should not be interpreted immediately as "physical has many more meaningful topics." It may reflect metadata practices, compound subject strings, punctuation, separators, granularity, or the way the field was split.

Use the surprising result to discuss method choice before implementing a larger analysis:

```
The physical sample has far more unique subject values than the digital sample.
This seems like a text-cleaning or lightweight NLP problem. Do not implement
anything yet. What simpler libraries or methods could help us inspect whether
these are near-duplicates, formatting variants, or genuinely granular subjects?
```

The useful teaching point is the decision process. A library such as RapidFuzz can score string similarity, and scikit-learn can support TF-IDF comparisons or clustering, but neither one automatically solves the interpretation problem. The analyst still needs to decide how subjects should be parsed, normalized, reviewed, and possibly grouped.

You can have the agent write a handoff prompt for a future agent:

```
Write a handoff prompt for another coding agent to investigate the subjects
field as a keyword-analysis problem. The prompt should ask the agent to profile
the field, compare raw and normalized unique counts, identify examples of
near-duplicate subject values, recommend a simple method before using an LLM,
and avoid large-scale canonicalization unless explicitly asked.
```

If you start a new thread with `/new`, you can use this prompt to perform new analysis in a new notebook. (You can also keep going with the same agent, this just demonstrates hand off)

Summary

The workflow completes one full iteration of the cycle: an initial broad prompt produced results without an artifact; reviewing the output produced project rules; the data structure was established before analysis; and the correct file was used to answer the question with traceable code.

```
Run a prompt.
Review what Codex did.
Define rules early.
Apply the rules while continuing the analysis.
Add rules as further issues appear.
Compare results before and after, and continue.
```

Key points

- The aggregate file answers questions about total checkout volume.
- The title-level sample files answer more granular questions about titles, subjects, material types, and publishers.
- Codex can help compare analysis approaches before writing code or installing anything.
- A coding agent can do much of the analytical work, but the person interpreting the results must confirm that the output is understandable, reviewable, and traceable.

Managing Context in Coding Agent Conversations

Codex can appear to remember the whole project. It may refer to files it inspected earlier, errors it saw, code it edited, or decisions you made several prompts ago.

That behavior is useful, but it can also hide an important boundary. Codex does not independently remember the project. Each response is generated from the context the application gives the model: project instructions, conversation history, file excerpts, tool results, command output, and the latest request.

For analytics work, the practical question is not only what to ask next. It is also what should stay in the conversation and what should be written into the project.

The central distinction

Use the conversation for **working context**.

Use project files for **saved project context**.

Working context is the temporary material Codex needs to continue the current task. It includes the current conversation, recent errors, command output, notebook cells, file excerpts, and decisions made during the session.

Saved project context is different. It is information written into files so that future humans and future Codex sessions can find it. The notebook, `AGENTS.md`, `README.md`, `data/README.md`, scripts, tests, and short decision notes can all serve this purpose.

The conversation can help discover facts. The project files should become the place where verified facts live.

Why this matters

A short prompt late in a session can depend on a long history of work:

```
Make the chart clearer.
```

That prompt may depend on earlier decisions about which CSV contains full checkout totals, which files are samples, whether 2026 is partial, and what the notebook section is trying to show. If those facts live only in the conversation, Codex may need to carry them forward, summarize them, or rediscover them later.

That can waste time, tokens, and attention. More importantly, it can make the analysis harder to review. A result is not very traceable if the reason for it is buried in an old chat turn instead of recorded near the notebook code or in a data note.

In the checkout project, suppose Codex learns that `combined_checkout_totals_by_month_usageclass.csv` is monthly aggregate data, that the two title-level files are balanced samples, that the aggregate file should be used for full checkout volume trends, and that 2026 is partial. Those facts affect which analysis is valid. They should not live only in the chat. They belong in the notebook, `data/README.md`, or another project note.

Manage context while you work

The goal is not to keep conversations short at all costs. A long session can be useful when it stays focused on one task. The problem is old context that no longer helps with the current task.

Use the same conversation when the previous context is still helping: for example, while working through one bug, one analysis question, one notebook section, one visualization, or one focused data investigation.

Start fresh when the phase of work changes and the important facts have been saved. This is often useful after broad data profiling, before starting a new analysis question, after a long debugging sequence, or when Codex keeps referring to an old assumption.

If the conversation is long but still relevant, `/compact` can help by replacing older history with a summary. Compaction is useful, but it is lossy. A summary may lose exact file names, column names, code details, or the reason a decision was made. Compaction is not a substitute for saving important context in project files.

Practical rule

Before starting fresh or compacting, ask: what would the next analyst need to know?

Choose model and reasoning level

Context management is not only about what you put in the prompt. It is also about choosing the right model behavior for the work.

A stronger reasoning model or a higher reasoning setting can be useful when the task has many dependencies: debugging a notebook failure, comparing possible analysis plans, deciding which dataset supports a question, or reviewing whether a conclusion follows from the code. Those tasks require the model to hold several constraints in mind and test alternatives before it acts.

That extra reasoning is not free. Reasoning uses tokens, takes time, and still depends on the quality of the context you provide. If the model has the wrong file, stale assumptions, or a vague goal, more reasoning can produce a more elaborate wrong answer.

Use a faster or lower-reasoning setting when the task is narrow and mechanical: fixing a typo, reformatting a small table, renaming a chart title, or applying a specific edit you can describe clearly.

Use a stronger or higher-reasoning setting when the task requires judgment: choosing an analysis approach, tracing a result back to source files, debugging an error with several possible causes, or reviewing whether a notebook section is understandable, reviewable, and traceable.

Practical rule

First improve the context. Then increase reasoning if the task still requires more planning, comparison, or debugging.

Use `/model` is available, use it to inspect or change the current model for the session. Model names and available reasoning settings can change, so do not memorize a single model name as the lesson. The transferable skill is knowing what kind of work you are asking the model to do.

Useful Codex commands

In the Codex CLI, type `/` to see available slash commands. The exact list may change, so use `/help` as the source of truth.

For context management, `/status` is useful when you want to check the current session state. `/compact` is useful when the conversation is long but still relevant. `/new` is useful when old context is no longer helping. `/resume` and `/fork` are useful when you intentionally want to return to or branch from a saved conversation.

These commands manage the conversation. They do not replace good project records.

Optional: run a usage report

You can download a small analysis script that summarizes recent Codex sessions from your local `~/.codex` folder. From the project folder, create a `scripts/` directory and retrieve the script with `curl`:

```
mkdir -p scripts
curl -L \
  -o scripts/analyze_codex.py \
 ␣
↪https://gist.githubusercontent.com/janash/d057c92b75f5a3f7990eedf3ecb0927a/raw/01958db1bcc0360b6f22
```

Then run it:

```
python scripts/analyze_codex.py --days 2
```

The script writes a `usage_reports/` folder with a Markdown summary, an HTML report, and CSV files. Use the report as a reflection tool. Look for sessions that mixed several phases of work, repeatedly inspected the same files, or produced large outputs where a project note or fresh session might have helped.

Do not treat the report as a perfect explanation of billing. Use it to notice patterns in how you worked.

Key points

- Codex conversations are useful working context, not saved project context.
- Verified facts, decisions, caveats, and reusable commands should be written into project files.
- Start fresh when old context stops helping, but only after the important facts have been saved.
- Match the model and reasoning level to the task: use more reasoning for planning, debugging, and review; use less for narrow mechanical edits.
- Ask for focused evidence instead of large outputs.

Homework: Extend the Keyword Analysis

This homework gives you another chance to practice the Session 1 workflow on a messy field: the `subjects` values in the Seattle Public Library title-level sample data.

The goal is not to produce a polished report or a final subject taxonomy. The goal is to practice using Codex to inspect data, compare approaches, and decide what kind of method is appropriate before trusting an automated result.

Choose one path

If you did not complete the subject-tag stretch exercise during the session, start with *the subject-tag stretch exercise*.

```
Do the checked out materials have subject tags in any dataset? Check the raw
CSV headers and report which files include the field.
```

Then ask Codex to add a small notebook section that profiles the `subjects` field in the digital and physical title-level samples:

```
Add a cell to the notebook that summarizes the subjects field for the digital
and physical title-level sample files. Count the number of unique subject
values in each dataset and show a small sample of values. Keep the code simple
and explain any parsing assumptions.
```

If the result is surprising, pause before asking Codex to implement a larger solution. Use the same method-choice prompt from the stretch exercise:

```
The physical sample has far more unique subject values than the digital sample.
This seems like a text-cleaning or lightweight NLP problem. Do not implement
anything yet. What simpler libraries or methods could help us inspect whether
these are near-duplicates, formatting variants, or genuinely granular subjects?
```

You can also ask Codex to write a handoff prompt, then start a new thread with `/new` and use that prompt to perform new analysis in a new notebook:

```
Write a handoff prompt for another coding agent to investigate the subjects
field as a keyword-analysis problem. The prompt should ask the agent to profile
the field, compare raw and normalized unique counts, identify examples of
near-duplicate subject values, recommend a simple method before using an LLM,
and avoid large-scale canonicalization unless explicitly asked.
```

If you already completed the subject-tag stretch exercise, extend it by trying a different model, reasoning setting, or method. For example, you might compare:

- a faster model and a stronger reasoning model

- a simple normalization-only approach and a fuzzy-matching approach
- RapidFuzz and scikit-learn TF-IDF
- an LLM-generated grouping proposal and a simpler string-similarity method

Questions to investigate

Use the exercise to answer practical questions about the field:

- What does the `subjects` field appear to contain?
- How different are the raw unique counts for digital and physical samples?
- Does simple normalization reduce the number of distinct values?
- Are there obvious near-duplicates, formatting variants, or compound subject strings?
- Does the large physical count look like meaningful topical variety, metadata granularity, parsing noise, or a mixture?
- What would you trust Codex or another model to do automatically?
- What would still require human review?

Session 1 Q&A

This page contains answers to questions left on sticky notes in Session 1.

How do I analyze my own data with Codex?

Codex works from the folder where you start it. That folder is the project workspace Codex can inspect, edit, and run commands in.

To analyze your own data, make a project folder first. Put your data files in that folder, then start Codex from inside the project folder. A useful structure is to keep the raw data in its own folder and keep notebooks, scripts, project notes, and instructions one level above it:

```
my-analysis-project/
├── .venv/
├── AGENTS.md
├── README.md
├── requirements.txt
├── analysis.ipynb
├── scripts/
└── data/
    ├── my_file.csv
    └── another_file.xlsx
```

This structure gives Codex a clear boundary: the project folder is the working space, and `data/` is where the source files live. It also keeps analysis files from getting mixed into the raw data folder.

Before analyzing the data, ask Codex to create a project-local Python virtual environment. A virtual environment keeps the Python packages for this project separate from the rest of your computer. For data analysis, the common "scientific Python stack" includes packages such as `pandas`, `numpy`, `scipy`, `matplotlib`, and Jupyter tools.

Ask Codex to set that up inside the project folder:

```
Please create a project-local Python virtual environment in .venv, install the
basic scientific Python stack for data analysis, and save the package list in
```

(continues on next page)

(continued from previous page)

```
requirements.txt. Do not install packages into the base Python or base conda
environment.
```

After Codex finishes, review what it did. You should see a `.venv/` folder and a `requirements.txt` file. The virtual environment can be deleted and recreated, but the requirements file should stay with the project so the setup can be repeated later. If you use Git, commit `requirements.txt`, not the `.venv/` folder.

Further reading: Python's documentation explains virtual environments, and the SciPy project provides installation guidance for the scientific Python ecosystem.

How can I move my files to WSL?

If you are using WSL on Windows, the easiest way to move files into your Linux project folder is often to open that folder in Windows File Explorer.

From the WSL terminal, move into your project folder and run:

```
explorer.exe .
```

The `.` means "this folder." File Explorer will open the current WSL directory, and you can drag files into the project just like any other Windows folder.

After moving files, return to the WSL terminal and run `ls` to confirm they are in the project folder.

Once the files are in place, open a terminal in the project folder and start Codex there. Then ask Codex to inspect the data before analyzing it:

```
Please inspect the files in data/. For each file, report the row count, column
names, data types, non-null counts, and the first few rows. Do not make plots
yet.
```

After Codex profiles the data, you can ask it to add project-specific guidance to `AGENTS.md`. For example, if one file is sampled and another is complete, write that down so future Codex sessions do not have to rediscover it.

Does my data need to be formatted a certain way?

For this course, the important question is not whether the data is formatted for an AI model. We are using AI-assisted programming for data analysis. That means Codex is helping you write and run code, and the code still needs to read your files through normal programming libraries.

Your data will be easiest to analyze when it is in a regular, tabular format that tools like Python, pandas, R, Excel, Tableau, or databases can read reliably. CSV, Excel, Parquet, JSON, and database tables can all work, depending on the project. The key is consistency:

- one row should represent a consistent unit
- column names should be present and meaningful
- values in a column should have a consistent meaning
- dates, numbers, categories, and missing values should be represented consistently
- repeated tables, notes, merged cells, subtotals, and visual-only formatting should not be mixed into the analysis data

If your data is not already consistent, do not overwrite the original file. Keep the raw data, create a cleaned or regularly formatted version, and save the code that performs the conversion.

A useful project structure is:

```
my-analysis-project/
├── AGENTS.md
├── README.md
├── analysis.ipynb
├── scripts/
│   └── prepare_data.py
└── data/
    ├── raw/
    │   └── original_file.xlsx
    └── processed/
        └── analysis_ready_file.csv
```

You can use Codex to help write the conversion script:

```
Please inspect data/raw/original_file.xlsx and write a script that converts it
to a regular analysis-ready CSV in data/processed/. Preserve the raw file. Put
the conversion code in scripts/prepare_data.py and document any assumptions.
```

This gives you three things a reviewer can check: the original data, the analysis-ready data, and the script that explains how one became the other.

What are the pros and cons of using the command-line Codex versus the app version?

Both versions are Codex. The main difference is the working environment around the agent.

The command-line version runs in your terminal. It is a good fit when you are already working in a project folder, using shell commands, running Python scripts, starting JupyterLab, checking files with `ls`, or using Git from the command line. For this course, the CLI makes the project boundary very visible: Codex starts in the directory you choose, reads files from that workspace, and runs commands there.

The tradeoff is that the CLI expects you to be comfortable with terminal workflow. You need to know where you are in the filesystem, how to move into the right folder, and how to interpret command output. If the terminal is new to you, this can make the first few steps feel less familiar.

The Codex app gives you a desktop interface for choosing projects, working with threads, seeing diffs, using built-in Git tools, and running tasks in local, worktree, or cloud mode. It also has an integrated terminal, so you can still run project commands when needed. The app can be easier when you want a more visual place to manage threads, review changes, or work on multiple projects.

The tradeoff is that the app can hide some of the filesystem mechanics that are useful to learn. For data analysis, it is still important to understand which project folder Codex is working in and where your data files are stored.

For this workshop, the command line is useful because it teaches the mental model:

- make a project folder
- put the data inside the project
- start Codex from that project
- inspect the data before analyzing it
- save scripts, notebooks, and project notes alongside the data

After that model is clear, the app can be a convenient interface for the same kind of work.

Further reading: OpenAI's Codex docs describe the Codex CLI, the Codex app, and Codex app features.

Should we worry about token usage for this course?

For this course, the main goal is to get comfortable using Codex for real data analysis work. OIT is supporting your use during the course, so you should take advantage of the opportunity to experiment as much as your available limits and your own comfort allow.

That experimentation is part of the learning. Try asking Codex for a plan before implementation. Try asking it to explain code it wrote. Try asking it to revise a notebook section so the result is easier to review. Try starting a new thread after saving project context. These repetitions are how the workflow becomes less mysterious.

At the same time, comfort matters. Some people may want to be mindful of usage limits, cost, or the energy use associated with generative AI systems. That is a reasonable concern. You do not need to use Codex for every small task if that does not feel right to you.

A practical balance is:

- use Codex enough to learn what it is good at and where you still need review
- ask focused questions instead of requesting huge unfocused outputs
- save useful findings in project files so you do not have to rediscover them
- stop when the tool is no longer helping your understanding

In short: be thoughtful, but do not be timid. This course is a supported chance to try the tool, make mistakes, compare prompts, and build judgment about when AI-assisted programming helps your analysis.

Which slash commands and shortcuts should I learn first?

You do not need to memorize every Codex command. For this course, slash commands are useful because they help you steer the session without changing the main task.

Start with these:

- `/status`: check the current session, including model, permissions, context, and limits
- `/plan`: ask Codex to plan before changing files or writing code
- `/new`: start a fresh conversation in the same project when the task changes
- `/compact`: summarize a long conversation so the useful context is preserved
- `/mention`: attach a specific file or folder to the conversation
- `/model`: inspect or change the model when the task needs a different level of reasoning
- `/permissions`: review or change what Codex can do without asking first

The exact list can change, so the most important command is just `/`. Type `/` in Codex to see what is available in your current environment.

The `@` pattern is also worth learning. You can use `@` path autocomplete, or the `/mention` command, to point Codex at files that matter for the current question. For example:

```
Read @data/README.md and @analysis.ipynb. Then tell me whether the notebook
uses the right dataset for a monthly trend analysis.
```

This is not about using a special trick. It is about making context explicit. If a file matters, do not make Codex guess which file to inspect.

Further reading: OpenAI's Codex docs describe CLI slash commands and IDE slash commands.

4.3 Session 2: Context Management and Agent Skills

Session 2 continues the context-management thread from Session 1, then introduces agent skills as a way to make repeated workflows reusable.

The main practical shift is that participants should start moving from notebooks as the default working artifact toward Python source files for agent-heavy analysis. Notebooks are useful for teaching and review, but this course is building toward dashboards. For iterative analysis work, a plain `.py` file or a percent-cell `.py` file usually gives the agent a simpler artifact to edit, run, inspect, and revise.

Page	Questions	Objectives
Context Management Part 2	• How does the working file format change what Codex has to do?	• Explain why notebooks can create extra tool use and choose a better working artifact for agent-assisted analysis.
Agent Skills	• How can reusable instructions make agent workflows more consistent?	• Distinguish prompts, project rules, and skills, then create or evaluate a small analytics skill.

Context Management Part 2: Choose a Working File Format

Session 1 used a notebook because notebooks are a familiar interface for many analysts and learners. They make it easy to see code, outputs, and short explanations together while learning the agent-assisted workflow.

Session 2 shifts from the notebook as a familiar learning interface to the working file as part of context management. When Codex performs repeated analysis work, the file format affects what the agent must read, edit, validate, execute, and inspect.

Objectives

- explain why working file format affects agent tool use
- distinguish notebook and percent-cell Python files as agent working artifacts
- interpret token usage with attention to model calls, fresh input, and tool results
- choose an appropriate working artifact for agent-assisted analysis

This may be easy to believe intuitively. A notebook looks more complex than a Python script, and an `.ipynb` file contains more than source code. The mechanism matters because Codex does not only read the file once. During a task, it may inspect the file, edit it, run it, inspect outputs, repair errors, and use those tool results as context for later model calls.

Central Claim

The working artifact is part of context management because it changes the tool work Codex must do. More tool work can mean more model calls, more fresh input, more cumulative context, and more runtime.

Start With One Agent Turn

To see why file format can affect token usage, first identify the unit of work being repeated. A single user request can require several model calls and tool calls before the final response. Each model call receives context, decides a next step, and may ask a tool to inspect, edit, run, or validate something in the project.

An agent turn is more than a chat message. The model receives input, decides the next step, may call tools, receives tool results, and may repeat that loop before answering.

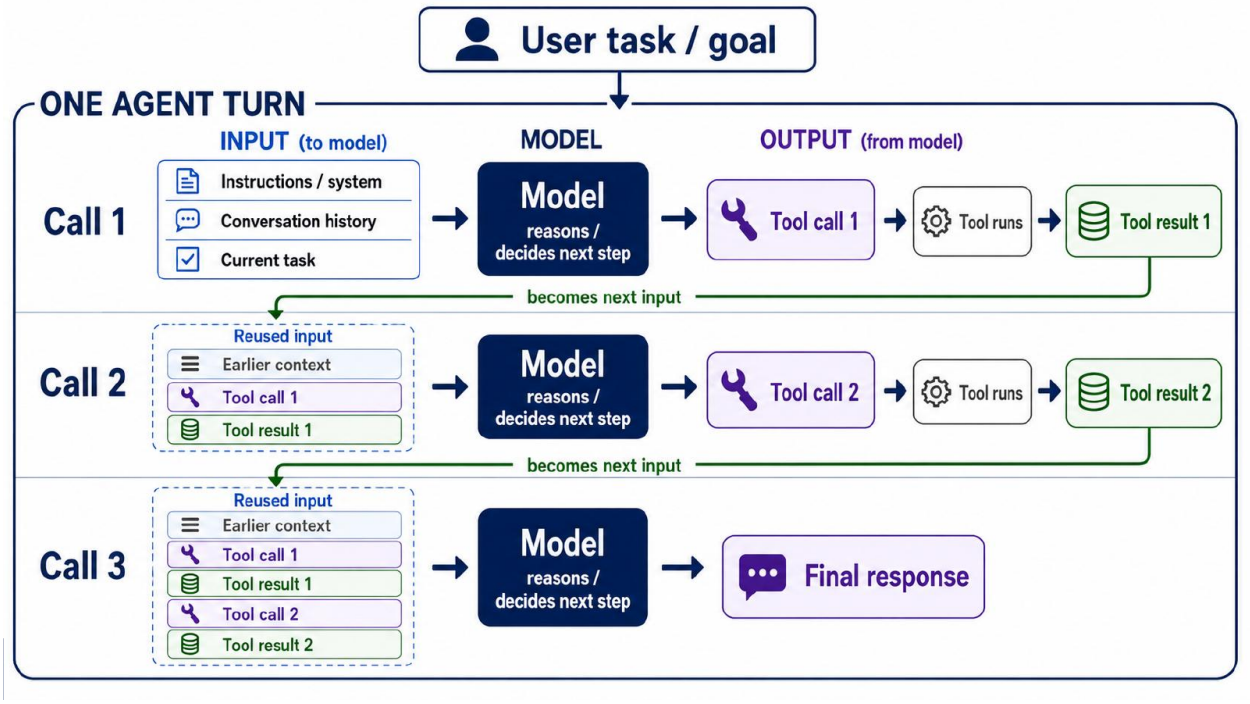

Fig. 3: One user request can produce several model calls. Tool results from one call can become input to the next call.

This matters because the working file is one of the objects the tools operate on. A file format that requires more inspection, validation, or output extraction can create more tool work during the same analysis task.

Why File Format Changes Tool Use

A Jupyter notebook is stored as JSON rather than as a simple script. An `.ipynb` file can contain code, markdown, outputs, execution counts, widget state, metadata, and nested structure that must remain valid.

A percent-cell `.py` file is plain Python text with cell markers. Tools such as VS Code and Jupyter can treat the markers as cells, but the file remains easy to edit as source code.

A notebook may require the agent to do extra tool work:

- preserve valid JSON structure
- add or update cells without corrupting the file
- execute the notebook
- inspect saved cell outputs
- decide which outputs matter
- remove or avoid stale outputs
- validate notebook structure after edits

A `.py` file gives the agent a simpler source artifact:

```
# %%
import pandas as pd

checkout_totals = pd.read_csv("data/combined_checkout_totals_by_month_usageclass.csv")
```

(continues on next page)

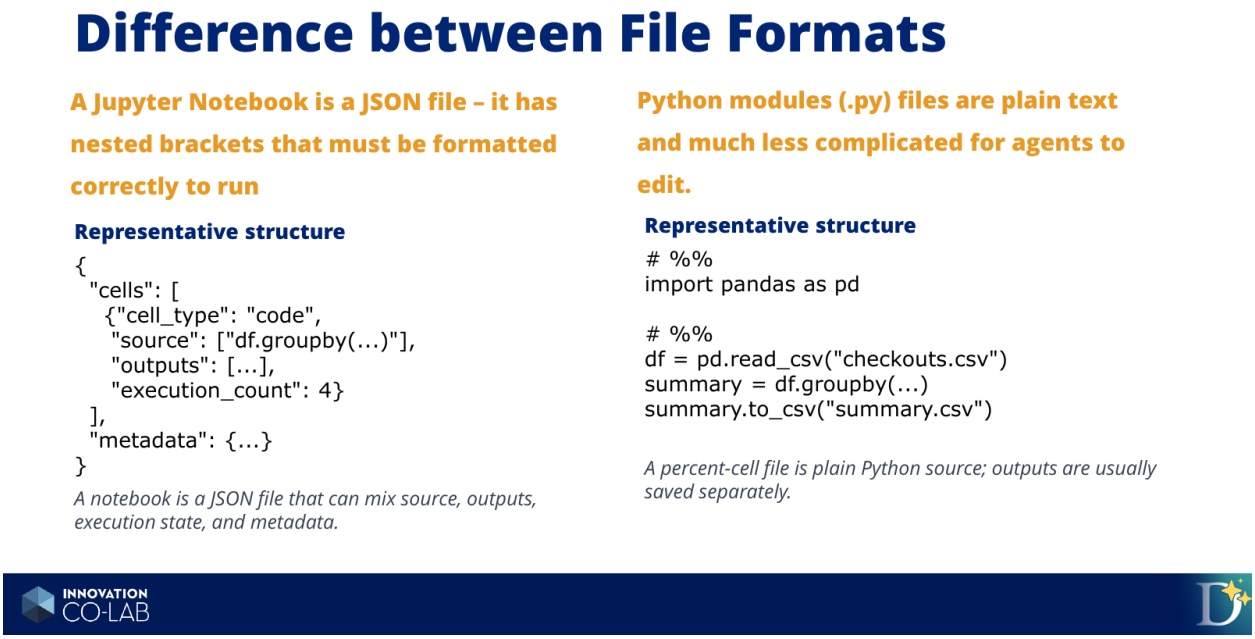

Fig. 4: A notebook mixes source, outputs, execution state, and metadata. A percent-cell Python file keeps the working source as plain text.

(continued from previous page)

```
# %%
complete_years = checkout_totals[checkout_totals["checkout_year"] < 2026]
annual_totals = (
    complete_years
    .groupby(["checkout_year", "usageclass"], as_index=False)["total_checkouts"]
    .sum()
)

# %%
annual_totals.to_csv("outputs/annual_checkout_totals.csv", index=False)
```

The `# %%` markers preserve a cell-like workflow, but the agent is editing ordinary Python text. Outputs can be saved separately as CSV files, images, markdown summaries, or dashboard-ready extracts.

How Extra Tool Work Becomes Extra Context

The file-format difference matters because tool results can become later input. If an agent validates notebook JSON, extracts cell output, receives an execution error, or inspects a generated artifact, that result can become part of the working context for the next model call.

More tool work can therefore mean:

- more model calls
- more command output and tool results
- more fresh input

- more cumulative context
- more chances to inspect the same information repeatedly

For context management, prompt length is only one part of the problem. The working artifact also determines how much tool work the agent must perform.

Why Token Type Matters

Token usage is not a single kind of activity. Some tokens are material the model reads, some are material the model writes, and some input tokens are repeated context that the system can cache. These distinctions matter because agent workflows often differ in how much new file content, command output, tool result text, and conversation history they add over time.

A **token** is a small piece of text or data the model reads or writes. When reviewing agent usage, separate these categories:

- **Input tokens**: what the model receives, including instructions, conversation history, file contents, command output, errors, and tool results.
- **Output tokens**: what the model generates, including explanations, code, edits, and final responses.
- **Cached input**: repeated context the system recognizes and can reuse, such as stable instructions or earlier conversation context.
- **Fresh input**: new context added on a turn, such as changed files, fresh command output, new errors, or newly inspected results.

Cached input is still context, but it is less expensive than fresh input. Fresh input is especially important for understanding workflow friction because it often reflects new material the agent had to inspect: a changed file, a notebook output, an error message, or a tool result.

A File-Format Experiment

The mechanism above suggests that notebooks may create more tool work for coding agents. To test that idea, we ran a paired file-format experiment.

In the experiment, agents performed similar Seattle Public Library checkout analyses. The analysis question was the same: compare physical and digital borrowing trends over time. The main difference was the working artifact:

- one workflow used an `.ipynb` notebook
- one workflow used a percent-cell `.py` file

The controlled comparison is the easiest part to interpret. Two agents were given the same data, the same analysis question, and similar deliverable requirements. The intended difference was the durable working source:

Workflow	Total tokens	Fresh input	Model calls	Runtime
Direct `.ipynb`	1,395,748	129,036	24	8m 30s
Percent-cell `.py`	719,688	51,000	14	3m 40s

The exact multiplier is less important than the mechanism:

> The notebook workflow required more model calls and more tool work. That extra work accumulated more context.

The tool-call categories showed the source of the difference. The direct notebook workflow included notebook-specific work such as structure validation and output extraction. The percent-cell workflow still required execution and output inspection, but it avoided much of the notebook-specific handling.

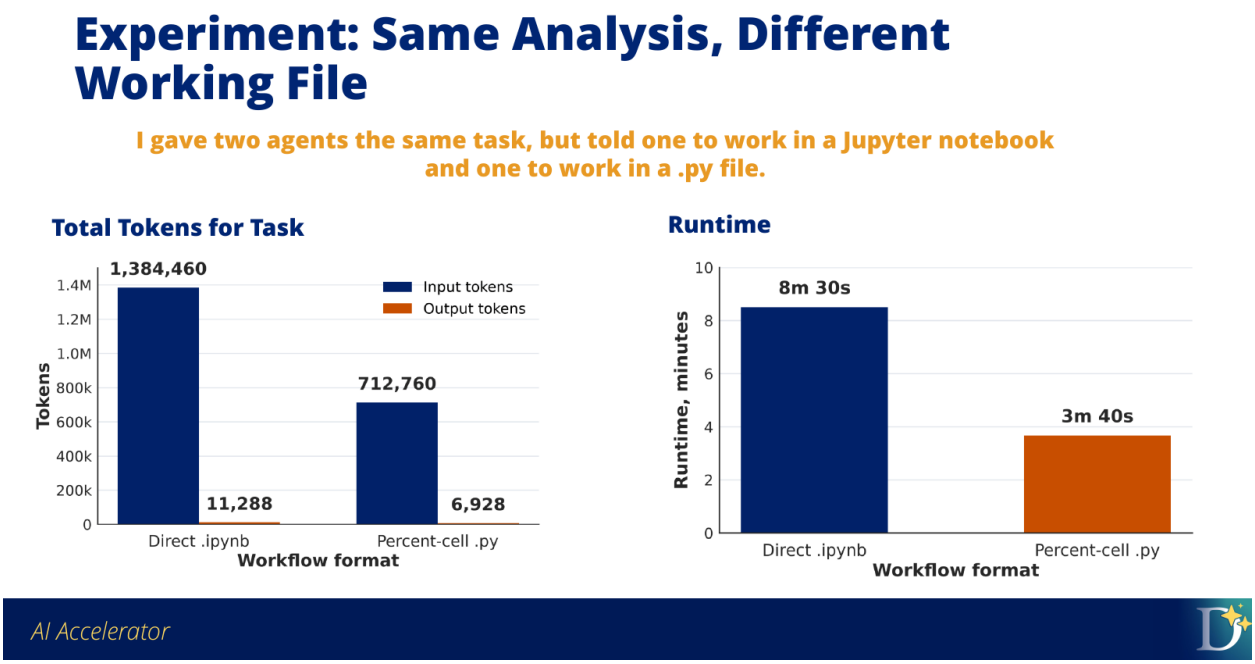

Fig. 5: In the controlled comparison, the notebook workflow used more total tokens and took longer.

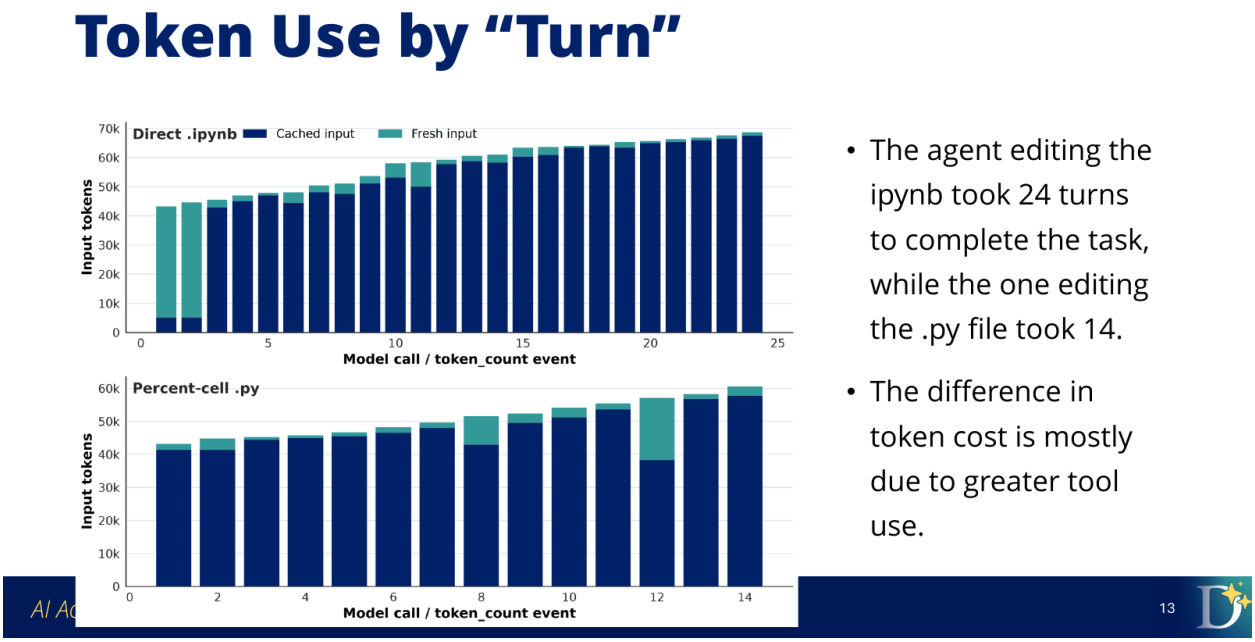

Fig. 6: The notebook workflow took more model calls. Each call is another chance for context, tool output, and file state to enter the next step.

We also ran a less constrained agent-native comparison. That comparison was noisier, but it pointed in the same direction: the notebook path again used more total tokens, more model calls, and more runtime. That consistency is why the recommendation is practical even though the exact multiplier is not universal.

More detail

For the complete charts, tool-call categories, cache notes, and starting prompts, see the full file-format evaluation report.

What This Means For Your Work

For the rest of this course, the destination is a dashboard or shareable analysis artifact. That changes the default working pattern.

Working Recommendation

For agent-heavy analysis, prefer a plain `.py` file or a percent-cell `.py` file as the working artifact. Use notebooks only when notebook-style teaching, presentation, or review is part of the task.

Use a `.py` or percent-cell `.py` file when:

- Codex will iterate through several edit-run-inspect cycles.
- You expect to create dashboard-ready data extracts.
- You want cleaner diffs and easier review.
- You want outputs saved as separate files.
- You do not need notebook-style explanation beside every cell.

Use a notebook intentionally when:

- the notebook itself is the teaching or review artifact
- you need rich inline display while presenting
- collaborators expect to read the analysis as a notebook
- the value of notebook presentation outweighs the extra agent workflow cost

The goal is to choose a working artifact that matches the job, so the agent spends its effort on analysis rather than artifact maintenance.

Practical Rule

Use Python source while Codex is iterating on the analysis. Use a notebook or report when the analysis needs to be read as a narrative artifact.

Key points

- File format is part of context management.
- Notebooks can require extra structure validation and output inspection.
- Extra tool work can increase model calls, fresh input, runtime, and total tokens.

- For agent-heavy analysis in this course, prefer `.py` or percent-cell `.py` as the working artifact.
- Use notebooks when the notebook format is part of the task.

Agent Skills

In Session 1, you used prompts and `AGENTS.md` to shape how Codex worked in one project. In the previous lesson, you saw that the working file format also shapes the agent's behavior.

Skills solve a related problem: repeated instructions should not live only in the conversation.

When the same guidance appears in many prompts, that guidance may belong in a skill.

The Problem Skills Solve

Suppose a project repeatedly needs figures for analysis presentations:

```
Use Duke colors. Make the figure readable on a PowerPoint slide. Use large text
and markers. Do not put a title on the figure. Save the figure as a high-quality
PNG.
```

The instruction can work in one prompt, but repeating it manually creates several problems. For example, you may forget part of the instruction, or it may be inconvenient if the task is complex.

A skill records that repeated workflow in a reusable instruction file.

Prompt, AGENTS.md, or Skill?

These three instruction types do different jobs:

Instruction type	Best use
Prompt	The immediate task: "Analyze this file" or "revise this chart."
`AGENTS.md`	Standing rules for one project: audience, file locations, raw-data rules, review expectations.
Skill	Reusable workflow guidance for a type of task across projects.

In Session 1, `AGENTS.md` told Codex how to work in the checkout-analysis project. A skill is more portable. It can define a reusable method for Duke-style figures, dashboard accessibility review, pedagogical explanations, or recurring data-validation checks.

What Is a Skill?

An agent skill is a reusable instruction set that extends an AI agent with specialized knowledge or a procedural workflow. In practice, a skill is usually a folder or repository containing:

- a `SKILL.md` file
- optional supporting files, such as templates, examples, scripts, or reference material

The core of a skill is `SKILL.md`. Supporting files are optional. For example, Jessica's pedagogy skill starts with metadata that lets an agent discover when the skill applies, followed by instructions for how to teach. For example, a skill on pedagogy might have the following instructions:

```
---
name: pedagogy
```

(continues on next page)

(continued from previous page)

```
description: "Use this skill when creating educational content that needs to␣
↪genuinely teach, not just inform."
---

These are loaded into agent context at start time, and the agent may read the rest of␣
↪the file when it encounters a relevant task.

## How Skills Are Loaded

Agents do not need to load every full skill into context all the time. Skills use a␣
↪pattern called **progressive disclosure**. The [Agent Skills␣
↪documentation](https://agentskills.io/home) describes this in three stages:

- **Discovery**: the agent sees the name and description, enough to know when the␣
↪skill might be relevant.
- **Activation**: when a task matches the skill, the agent reads the full `SKILL.md`.
- **Execution**: the agent follows the instructions and may load supporting files or␣
↪run bundled scripts if the skill calls for them.

A skill works because the agent can discover it, read it when relevant, and apply the␣
↪written instructions to the current task.

You can also explicitly attach a skill in a prompt:

```text
$pedagogy Draft this lesson so students understand why the workflow works, not
just what commands to run.
```

The `$skill_name` pattern tells the agent which reusable instruction set should shape the response.

Where Skills Live

Codex and other tools look for skills in well-known locations. Common locations include:

Scope	Example locations
Global skills	`~/.codex/skills`, `~/.agents/skills`
Project skills	`.codex/skills`, `.agents/skills`

Global skills are available across projects. Project skills travel with a repository and are useful when everyone working in the project should share the same workflow guidance.

Use a project skill when the instruction is part of how this project works. Use a global skill when the instruction is part of how you personally work across many projects.

Skill Effects

The same prompt can produce different outputs when different skills are active. In the deck example below, a project used skills to steer an AI art system connected to a physical drawing device. The prompt stayed the same, but the skill changed the method and style.

For analytics, the difference may be less visually dramatic but operationally useful. A skill can direct Codex to:

- always profile data before modeling
```

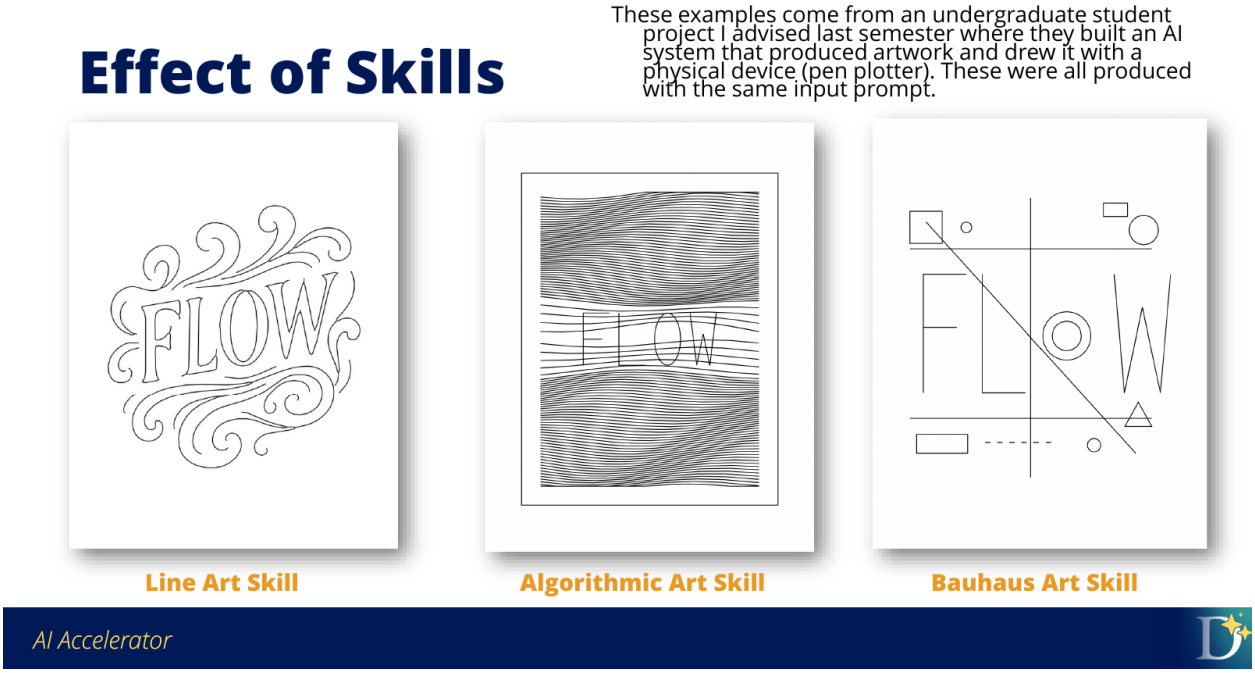

Fig. 7: Skills can make the agent apply a specified method to the same request.

- use a specific chart style
- validate dashboard-ready output files
- check accessibility issues in a chart
- produce a stakeholder-ready summary format

Skills Repositories

Skills can be shared through Git repositories. These collections provide examples to inspect, install, or adapt:

- Duke AI skills collection
- OpenAI skills collection
- Anthropic skills collection

Review a skill before using it. A skill can include instructions, scripts, templates, and other supporting files, so it should be treated as project code rather than only as prompt text.

Installing or Creating Skills

Some skills can be installed from a Git repository with `npx skills`:

```{admonition} If `npx` is missing :class: tip

`npx` is included with modern versions of `npm`. Check that Node.js, `npm`, and `npx` are available:

```
node --version
npm --version
npx --version
```
```

If one of these commands is missing, install Node.js and `npm` using the setup instructions for your operating system, then run the checks again.

```
```bash
npx skills install SKILL_REPO
```

If a repository contains multiple skills, specify the one you want:

```
npx skills install SKILL_REPO --skill SKILL_NAME
```

You can also ask Codex to help create a skill with `$skill-creator`:

```
$skill-creator I want to create a skill for producing figures when I'm doing
data analysis. I want to use the Duke color scheme for my figures, and I want
the output to be high quality and appropriate for PowerPoint. The text and
markers should be large and readable, and figures should not have titles.
```

That prompt describes the reusable workflow standard that the skill should capture.

Test a Skill Like an Analyst

Evaluate a skill before relying on it.

Use a before-and-after comparison:

1. Ask Codex to perform the task without the skill.
2. Save or inspect the output.
3. Ask Codex to perform a similar task with the skill.
4. Compare the outputs against the behavior the skill was supposed to produce.
5. Revise the skill if the difference is not clear enough.

For the Duke figure example, you might check:

- Are the colors appropriate?
- Is the text readable on a slide?
- Are markers and lines large enough?
- Did the figure avoid unnecessary titles?
- Was the output saved at useful resolution?
- Would the figure work in a dashboard presentation?

This evaluation checks whether the written skill produces the intended behavior.

Key points

- Prompts guide the immediate task.
- `AGENTS.md` guides one project.
- Skills guide reusable workflows across tasks or projects.
- A skill is usually `SKILL.md` plus optional supporting files.
- Skills are loaded through discovery, activation, and execution.
```

- Test a skill by comparing behavior before and after using it.

4.4 Session 3: From Analysis to Dashboard

Session 3 moves from reusable agent workflows into dashboard delivery. The central shift is that analysis outputs become a contract between Python work and the dashboard layer.

Participants examine how the Seattle Public Library teaching dataset was built, then use percent-cell Python files to generate machine-readable data files, preserve provenance, and prepare outputs that a browser dashboard can read directly. Multi-agent workflows are useful here when the parent agent keeps ownership of the plan while smaller subagents handle bounded investigation or implementation tasks.

Page	Questions	Objectives
Building Datasets with Codex	• How do you turn a public data source into a usable teaching dataset? • When should you use complete aggregate files or sampled detail files? • How do sampling choices shape the claims learners can make?	• Investigate a public API access pattern before retrieval. • Separate volume questions from composition questions. • Define row meaning, sampling strategy, reproducibility, and validation checks.
Multi-Agent Workflows	• What is a multi-agent or subagent workflow? • When should you use subagents? • How do you use subagents without losing control of the project?	• Distinguish parent, worker, and checker responsibilities. • Decide when a task is ready to delegate. • Write bounded subagent task contracts and review the results.
Build and Deploy a Static Dashboard: Prompt Notes	• How can Codex turn dashboard-ready CSV files into a static dashboard? • How can prompts change the chart, library, or dashboard component? • What steps publish a static dashboard with GitLab Pages?	• Use prompts to generate and revise a static HTML/CSS/JavaScript dashboard. • Keep dashboard data loaded from CSV files at runtime. • Follow the basic GitLab Pages deployment sequence for static assets.
Open Dataset Starting Points	• What public datasets could support a student dashboard? • Which sources have fields that support filtering, aggregation, maps, or time series? • How can you choose a dataset that leads to a clear dashboard question?	• Compare several public data sources for dashboard potential. • Select a dataset with a usable unit of analysis and meaningful dimensions. • Turn a dataset idea into a focused dashboard question.

Building Datasets with Codex

Analysis datasets rarely arrive in exactly the shape you need. A useful dataset is designed: an analyst decides what questions the data should support, what level of detail is needed, what tradeoffs are acceptable, and how the finished files should be regenerated and checked.

Codex can help with that work, but it should not replace the analyst's judgment. Use Codex to inspect sources, write retrieval scripts, create repeatable transformations, and validate outputs. Keep the analyst responsible for the questions, definitions, caveats, and claims the dataset should support.

This lesson uses the Seattle Public Library checkout data as a running example, but the workflow applies to many public datasets.

Objectives

- explain why dataset building starts with analysis questions
- inspect a public data source before downloading it
- distinguish complete aggregate files from sampled detail files
- choose a sampling strategy based on the claims the dataset should support
- use Codex to write a reproducible dataset-building script
- define validation checks and documentation before a dataset is ready for analysis

Start With Analysis Questions

The first dataset design decision is not how to download the data. It is what the dataset needs to help analysts answer.

For example, a checkout dataset might support questions like:

- How have digital and physical checkout totals changed over time?
- What kinds of titles, material types, or subjects appear in each checkout format?
- How did the mix of materials change across years?

Those questions require different kinds of files. Total checkout volume needs complete aggregate data. Material mix can often be explored with a sampled detail file. If the dataset tries to do everything in one file, it may become too large, too slow, or too easy to misinterpret.

> **Analyst responsibility**
>
> Codex can help inspect a source and write retrieval code, but the analyst has to define the questions, acceptable tradeoffs, row meaning, and claims the dataset should support.

Find the Working Data Access Point

After defining the analysis need, identify how software can access the source. Public data may be available as a direct download, database export, API endpoint, bulk ZIP file, or query service.

> **What is an API?**
>
> An **API**, or application programming interface, is a structured way for software to request information from another system. For a public dataset, an API endpoint is often a URL that returns data a script can query, filter, and save.

In the Seattle example, the working public endpoint was:

```
https://data.seattle.gov/resource/tmmm-ytt6.json
```

You can find the source dataset page for Seattle Public Library checkouts on Seattle Open Data. The source is `tmmm-ytt6`, "Checkouts by Title." It includes both `Digital` and `Physical` values in a `usageclass` field, which made it a good fit for analyzing changing checkout patterns.

> **Endpoint note**

Some data portals expose more than one URL pattern. The original Seattle link used the `api/v3/views/.../query.json` pattern. For unauthenticated retrieval, the script used the public Socrata SODA resource endpoint instead: `https://data.seattle.gov/resource/tmmm-ytt6.json`. Confirm the working public endpoint before asking Codex to write retrieval code.

Inspect the Source Before Downloading

Before pulling a large dataset, ask Codex to inspect what the source represents. The important questions are structural:

- What does one source row represent?
- Is the source already aggregated, or is it row-level event data?
- What time span does it cover?
- Which fields are available for filtering, grouping, or sorting?
- How large is the source?
- Can the endpoint support server-side filters or grouped queries?

In the Seattle source, one row is not one checkout. The source is already aggregated: one row represents a title/material/usage-class/month combination, with a `checkouts` count.

That row meaning changes the analysis. If a row has `checkouts = 47`, it is a title-month record summarizing 47 checkouts for that combination. Treating that row as one checkout would undercount activity and distort any interpretation of volume.

Design the Output Files

Once the source is understood, design the smaller files analysts will actually use. This is where the analysis questions become file contracts. You can work with Codex to establish this based on the available data and questions you want to ask.

Question type	Example	Useful output file
Volume	How did digital and physical checkout totals change over time?	Complete aggregate totals
Composition	What kinds of titles, material types, or subjects appear in each checkout format?	Sampled detail records

For the Seattle data, this led to a three-file design:

```
seattle-public-library/
  combined_checkout_totals_by_month_usageclass.csv
  digital_checkout_title_sample_balanced_250k.csv
  physical_checkout_title_sample_balanced_250k.csv
  README.md
```

The totals file preserves complete monthly counts by `usageclass`. The two sample files provide manageable title/month records for digital and physical exploration. The sample files have the same structure because they come from the same source and were split by `usageclass` after retrieval.

Choose a Sampling Strategy

Sampling is not just a file-size trick. A sampling strategy changes what claims analysts can make.

Sampling was necessary for the Seattle title/month records because the source contained far more rows than we could reasonably package in a normal repository or ask analysts to load during a short exercise. The design keeps the monthly totals complete, because those files are small after aggregation. It samples the title-level detail records, because that is the part of the source that is too large to distribute directly.

Several sampling strategies could support different goals:

- **Most recent rows**: useful for current-state inspection, weak for historical comparisons.
- **Simple fixed-size sample**: easy to create, but may overrepresent high-volume periods or common groups.
- **Proportional sample**: useful when the sample should resemble the full source population.
- **Balanced sample by year**: useful when analysts need enough rows from each year for comparison.
- **Separate balanced samples by group**: useful when comparing groups that have very different source volumes.

For the Seattle data, the final choice was a balanced sample by year and usage class:

```
2006–2025
20 complete years
250,000 Digital rows
250,000 Physical rows
12,500 rows per year per usageclass
```

The sample is balanced at the year and usage-class level, not at the month level. Each year gets 12,500 Digital rows and 12,500 Physical rows. Within that year, months with more source rows get more sampled rows than months with fewer source rows. For example, if July has twice as many title/month records as February for Physical checkouts in a given year, July receives about twice as many rows in that year's Physical sample.

The script also makes the sample method repeatable. Running the script again should apply the same year range, group definitions, and sampling rules, rather than creating an undocumented one-off extract.

Balanced sampling fit this workflow because analysts can compare patterns across years. A proportional sample would represent the overall source population more directly, but high-volume years would dominate the sample and lower-volume years would have fewer rows for comparison. The complete totals file preserves real volume, so the sample can focus on composition.

> **Sampling tradeoff**
>
> A balanced-by-year sample is useful for comparing composition across years. A proportional sample is useful for estimating the overall source population. The right choice depends on the question the dataset should support.

Ask Codex for a Reproducible Builder

Once the dataset design is clear, ask Codex for a script, not a one-time download. The script is the method. It should show the source endpoint, retrieval choices, filters, sampling design, output paths, and validation checks.

The key prompt is not simply:

```
Download this data.
```

A better prompt is:

```
Write a reproducible script that creates these analysis files from this public
endpoint. Use the approved file design, make the sampling method repeatable,
write partial outputs safely, and validate the final row counts.
```

The script should make the retrieval method visible without requiring analysts to repeat manual download steps. For the Seattle dataset, that meant querying the public endpoint, creating the approved aggregate and sample files, and writing outputs that matched the dataset design.

Why write a script?

A script turns dataset retrieval into a reviewable method. Reviewers can inspect the source URL, retrieval choices, sampling design, output paths, retry behavior, and validation checks.

Prompt Sequence

Do not ask Codex to inspect the source, design the dataset, and build the script all at once. Break the work into stages so you can review the decisions before Codex writes code.

First, ask Codex to inspect the source:

```
I want to build a reproducible analysis dataset from [SOURCE].

Inspect the source access pattern and metadata. Do not download the full dataset
yet. Tell me:
- what the source appears to contain
- what one row represents
- whether the source is row-level or already aggregated
- what fields are available for filtering or grouping
- how large the source appears to be
```

Then ask for a dataset design:

```
Using what you found, recommend a dataset design for these analysis questions:
- [QUESTION 1]
- [QUESTION 2]
- [QUESTION 3]
```

After you approve the design, ask for the builder script:

```
Write a script that can regenerate the approved files.
The script should use a repeatable retrieval method create a short README to␣
↪accompany the script.
```

This sequence separates source inspection, dataset design, and implementation.

Key Points

Key points

- Dataset building starts with analysis questions, not downloading.
- Public data retrieval starts with source inspection.

- Row meaning determines what the dataset can and cannot support.
- Complete aggregate files and sampled detail files serve different purposes.
- Sampling strategy changes what claims analysts can make.
- A reproducible script makes the dataset method inspectable and rerunnable.
- Long-running downloads need partial-file handling, retries, and validation.
- Minimal documentation should explain source, files, row meaning, sampling, and retrieval method.

Multi-Agent Workflows

In Session 2, you saw that Codex's working file format changes the amount of tool work it has to do. You also saw that reusable instructions can make repeated workflows more consistent.

In this lesson, you will use Codex subagents to divide a dashboard data-preparation project into separate analysis tasks. The example continues with the Seattle Public Library checkout data. The goal is to prepare the data files that a dashboard can read.

A **multi-agent workflow** uses one main agent to coordinate the work and one or more additional agents to handle focused assignments. The main Codex session is the **parent agent**. A **subagent** is a separate agent run that receives a specific task from the parent, works in its own context, and returns findings, file changes, command output, or a summary.

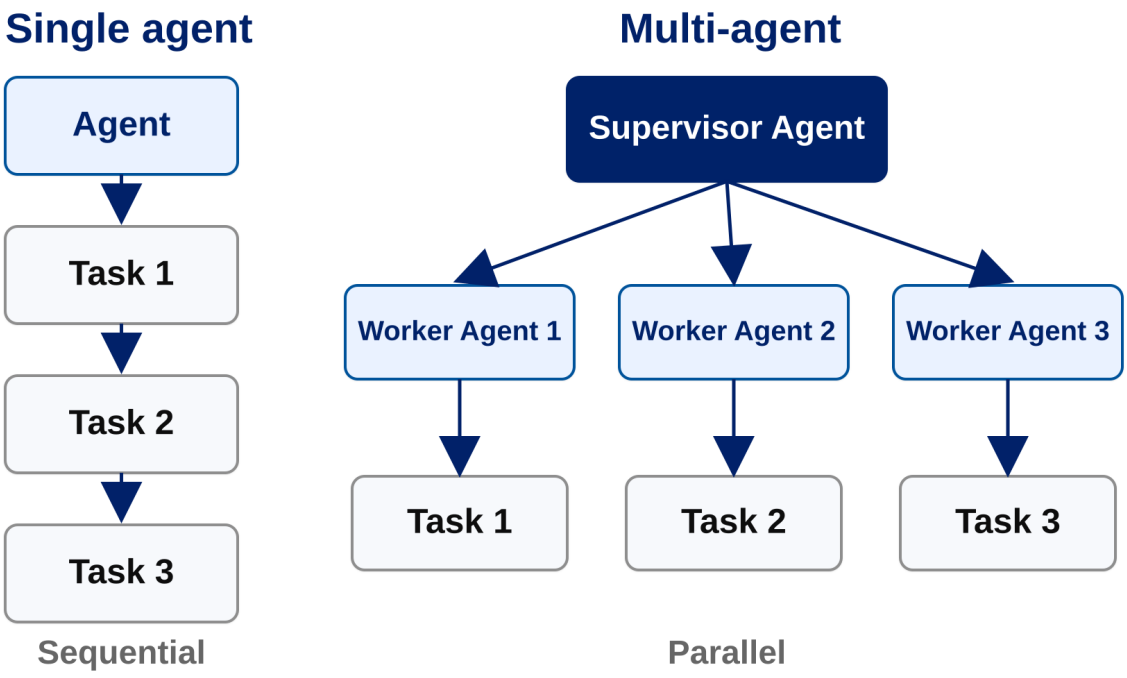

Fig. 8: A single agent works through the tasks in sequence. In a multi-agent workflow, the parent agent acts like the supervisor: it divides the work into bounded assignments, then reviews the results before integrating them.

Subagents are useful when a project can be divided into focused tasks that do not all require the same context at the same time. In analytics, those tasks might be separate analysis questions, separate validation checks, or separate parts of a dashboard build. In this lesson's dashboard example, one subagent can work on annual digital vs. physical share, another can work on sampled title popularity, and another can work on material-type shares. Each worker owns an analysis task: the percent-cell Python script, the CSV files that script writes, and any notes needed to interpret those files.

Objectives

- explain what a parent agent and subagent are
- use worker subagents to implement separate analysis tasks
- explain why a smaller worker model can be appropriate for bounded work
- specify machine-readable outputs for dashboard data preparation
- review worker outputs before dashboard code depends on them

Why Use Subagents Here?

A single Codex session can complete an entire analysis project. That is often the right choice when the task is small or when every step depends closely on the previous step.

The Seattle Library dashboard project has several analysis questions that can be developed separately:

1. How has digital vs. physical checkout share changed by year?
2. What was the most checked out sampled title each year?
3. What material types make up sampled digital and physical checkouts?

Those questions use related data, but they do not require one agent to hold every implementation detail at the same time. A parent agent can coordinate the project, while subagents handle focused pieces.

Codex supports subagents that are commonly used in focused roles:

- **Worker subagents** implement a bounded task. They need a clear write scope and a verification command.
- **Checker subagents** review outputs against a defined standard. They should usually be read-only.

> **Practical rule**
>
> Use worker subagents after the parent can describe the analysis task, source data, output file, and verification check.

Start With the Parent Agent

The parent agent should begin with the project rules and the assignment frame. In this workflow, the data source is already documented, so the parent should read the project rules and the Seattle Public Library data notes directly before defining the worker tasks.

This lesson also changes the working artifact. Earlier work used a notebook. For the dashboard workflow, the analysis should move into percent-cell Python files so the scripts can be rerun and can write standalone, machine-readable CSV outputs.

The following box gives a prompt that you should give to a Codex session. You should start Codex in your `student_setup` folder, and can paste it in (`cd analytics_accelerator/student_setup` then start codex)

```
We are preparing data for a Seattle Public Library checkout dashboard.

Read @seattle-public-library/README.md.

For this lesson, move the analysis out of notebooks and into percent-cell
Python scripts that write dashboard-ready CSV files.

Do not edit files yet. First, summarize the project rules and the data context:
```

(continues on next page)

(continued from previous page)

```
- what one row represents in each checkout file
- which source file supports each dashboard analysis
- which analyses use complete data or sampled data
- what caveats the worker prompts should preserve
```

Notice somethings about this prompt:

- We tell Codex we want to use `.py` files instead of notebooks.
- We use the `@` symbol to point to a particular file. This is a special command in Codex you can use to point to an existing project file.

The parent should extract rules like these:

- raw data files must not be modified
- accepted analysis belongs in percent-cell Python files
- each analysis script should write machine-readable CSV files
- each dashboard data file needs provenance

Launch Worker Subagents

After the parent summarizes the data context, ask it to launch worker subagents for the separate analyses. You can describe the goal in plain text. The parent should use the documented data context to turn that goal into specific worker assignments.

The workers need to produce data that the dashboard can reuse. That means the analysis should be separate from the visualization: percent-cell Python scripts define the calculations and write standalone CSV files with predictable columns, then the dashboard reads those CSV files.

Why a machine-readable file?

A dashboard should read data from a file, not from a chat summary or a chart-specific calculation hidden in the visualization code. A standalone CSV gives the project a reusable handoff: the analysis script creates the data, the CSV stores the result in rows and columns, and the dashboard renders that reviewed result. The same file can also be inspected, tested, reused in another view, or shared with someone who does not need the dashboard code.

```
Using the data context you summarized, launch worker subagents using gpt-5.4-mini.

Each agent should output data in a machine-readable format.

Create separate workers to (1) calculate annual digital vs. physical checkout share,␣
↪(2) identify the most checked out sampled titles by year, preserving all title␣
↪metadata with total checkouts, and (3) calculate sampled material-type shares by␣
↪year separately for digital and physical checkouts. Each worker should write␣
↪percent-cell Python code and write the CSV files from that code. Each analysis that␣
↪exists for both physical and digital should have separate files for physical and␣
↪digital. There should be five output files - one for physical vs digital share by␣
↪year, and two each for analysis (1) and (2).

After the agents are done, map the provenance of each file to scripts by documenting␣
↪in a markdown file in the dashboard data directory including how to regenerate the␣
↪data.
```

This prompt leaves the detailed decomposition to the parent. The parent should use the data documentation to decide which source files, grouping keys, output paths, caveats, and checks belong in each worker prompt. A worker assignment is ready when the parent can state the input data, the calculation, the output file, the expected columns, the provenance requirement, and the verification check.

The smaller model is appropriate here because the parent has already scoped the work. The `gpt-5.4-mini` workers are not discovering the project direction. They are executing bounded analysis tasks with source files, output expectations, and caveats supplied by the parent. Use a smaller worker model when the assignment has known inputs, known outputs, limited judgment calls, and a concrete verification command. Keep the parent or a stronger model responsible when the task still requires choosing the analysis strategy, resolving ambiguous data meaning, or making dashboard design tradeoffs.

Watch the Worker Threads

While the workers are running, you can inspect what each subagent is doing. In Codex, type:

```
/agents
```

Codex will show the active agent threads. Pick one worker thread and open it. Read through the thread the same way you would read the main Codex conversation: check which files the worker inspected, what commands it ran, what assumptions it made, and whether it is writing the kind of CSV file the dashboard needs.

You do not need to manage every step inside the worker thread. The parent agent is still coordinating the workflow, and you still control what gets accepted into the project. Reading a worker thread gives you visibility into the analysis while it is happening, and it gives you a chance to notice a wrong source file, a missing caveat, or an output shape that will be hard for the dashboard to use. If you see one of those problems, ask the parent to revise the worker assignment or review the worker output before integrating it.

What to watch for

The parent should ask workers for two deliverables: the analysis script and the CSV files written by that script. A prose summary can explain the result, but the dashboard needs the standalone files.

Use the Outputs in the Dashboard

Next, we will build a dashboard that uses this data. We will make a static dashboard (HTML, CSS, Javascript) that reads data from the CSVs this step produced.

The boundary is:

- analysis scripts define each calculation
- the scripts write CSV files
- CSV files are the dashboard inputs
- provenance notes explain how to interpret the CSV files
- dashboard code renders the reviewed outputs

We will ask Codex to make a new folder for us that contains only the data needed for the dashbord:

```
We will make a static dashboard for this project next. Please create a new folder for␣
↪the dashboard that contains the data files we've created in a data folder along␣
↪with the scripts to create the data files in a script folder.
```

Key points

- The parent agent is the main Codex session coordinating the project.
- A subagent is a separate agent run assigned to a bounded task.
- Workers own separate analysis questions.
- Each worker deliverable is the analysis script plus the CSV files that script writes.
- `gpt-5.4-mini` can be appropriate for worker tasks after the parent has tightly specified the contract.
- Machine-readable CSV files keep analysis outputs separate from visualization code and easier to reuse.
- Dashboard inputs should be reviewed before dashboard code depends on them.

Build and Deploy a Static Dashboard: Prompt Notes

These notes collect the prompts and deployment steps for the static dashboard lesson. The lesson follows *Multi-Agent Workflows*.

See the GitLab Pages deployment example: dashboard-starter-with-examples-8c1afe.pages.oit.duke.edu

Create a Dashboard

```
Create a static dashboard website using plain HTML, CSS, and JavaScript that loads␣
↪data directly at runtime from␣
↪dashboard_data/digital_material_type_shares_by_year.csv, do not embed it; use␣
↪Plotly.js to render exactly one interactive chart and one clearly written␣
↪interesting finding derived from the CSV data, make the page responsive and␣
↪visually polished.
```

Try a New Chart

```
Change to a stacked bar chart in Plotly.js
```

Try a New Library

- Plotly.js
- Apache ECharts
- Leaflet

```
Change to Apache Echarts
```

Try the Bar Race Chart

```
Change to a bar race chart
```

Create a Table

```
Create a table below the chart showing the most popular material type of each year␣
↪using AG Grid Community 32
```

Create a New Data Visualization

```
Create a new data visualization that loads data directly at runtime from␣
↪dashboard_data/digital_top_sampled_titles_by_year.csv, do not embed it; render␣
↪exactly one interactive chart and one clearly written interesting finding derived␣
↪from the CSV data. Adds it below the exisitng ones.
```

Create a Subagent

```
Create a repo-local subagent definition for CSV data visualization work, saved under␣
↪agents/. The subagent should accept a CSV path and output location, inspect the CSV␣
↪schema first, build a static responsive HTML/CSS/JS visualization, load the CSV␣
↪directly at runtime with fetch() and never embed data or derived full datasets, use␣
↪a browser visualization library such as Plotly.js, Apache ECharts, AG Grid␣
↪Community 32 for tables, or Leaflet for map-style views, render exactly one␣
↪interactive chart or requested table, compute exactly one clear finding from the␣
↪runtime-loaded CSV, keep edits scoped to the requested output files, and verify␣
↪syntax plus confirm the CSV is referenced rather than embedded.
```

Use Data Viz Accessibility Skill

```
use $data-viz-accessibility to audit the dashboard
```

Deploy on GitLab Pages

Set Up SSH Key

Ensure you have an SSH key added to your Duke OIT GitLab account inside https://gitlab.oit.duke.edu/. If you already have one, continue to the check below. If not, do the following:

1. Generate an SSH public/private key pair on your laptop as explained in the Duke OIT GitLab SSH documentation.
2. Upload the public half of that key pair to Duke OIT GitLab as explained in the add an SSH key documentation.

To check if you have set up your SSH key successfully, use the following command in your terminal:

```
ssh -T git@gitlab.oit.duke.edu
```

You should see the following message, with your NetID filled in for `NETID`:

```
Welcome to GitLab, @NETID!
```

Set Up the Repository

1. Move `index.html`, `index.css`, and `dashboard_data/` into a `public/` folder.
2. Create a new empty repository.
3. Fill in the form. Make sure the `README.md` option is not selected.
4. Add a new origin.

```
git init --initial-branch=main
git remote add origin2 git@gitlab.oit.duke.edu:ay114/mydashboard.git
```

(continues on next page)

(continued from previous page)

```
git add .
git commit -m "Initial commit"
git push --set-upstream origin2 main
```

5. Refresh the GitLab repository and confirm that you can see your code.

Set Up GitLab Pages

1. Visit **Settings -> General -> Visibility**.
2. Make sure the GitLab Pages setting is on.
3. Go to **Deploy -> Pages**, create a runner, and copy and paste the following setting:

```
image: alpine:latest

pages:
  stage: deploy
  script:
    - echo "Deploying static assets..."
  artifacts:
    paths:
      - public
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
```

4. Press commit. This step will generate a `.gitlab-ci.yml` file.
5. Check the pipeline.
6. View the pipeline and see it run.
7. When the pipeline succeeds, visit the Pages link.
8. Confirm that the dashboard is deployed.

Discuss

Open Dataset Starting Points

The sources below are starting points you might consider for your own dashboard analysis. Choose one you find interesting that gives you a question to investigate.

Area	Dataset / source	Why it could work
Local / Durham	Durham Trees & Planting Sites	Local, visual, structured; good for maps, filtering, dashboards, and ecosystem-benefit examples.
Parks / time series	NPS Visitor Use Statistics	Annual and monthly park visitation data; good for trends, seasonality, and comparison across places.
Higher ed	College Scorecard	Institutions, costs, programs, outcomes, debt, and earnings; strong for education dashboards.
Research / scholarly metadata	OpenAlex	Works, authors, institutions, sources, and topics; good for analyzing research trends.
Weather / climate	National Weather Service API	Forecasts, alerts, and observations; good for API and local dashboard examples.
Census / demographics	Census Data API	Population, housing, income, commuting, and education data; useful for maps and public data lessons.
Environment / health	CDC PLACES	Local health indicators by county, place, census tract, and ZIP Code Tabulation Area.
Arts / culture	Metropolitan Museum of Art Collection API	Public art collection data; good for search, filters, categories, dates, and maps.
Playful / animals	2018 Central Park Squirrel Census	Fun and approachable; good for categories, maps, filtering, and counts.
Playful / media	Bob Ross Painting Elements	Small and easy to understand; good for categorical summaries of trees, mountains, clouds, cabins, lakes, and other painting elements.

Before building, write down the dashboard question in one sentence. Then inspect the source enough to confirm the data has the fields, license, and access pattern your dashboard needs.